

Web and Data Engineering

Kapitel 3: Frontend-Technologien — HTML, JavaScript und CSS

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

Jede Web-Anwendung, die ein Nutzer sieht und bedient, besteht aus drei Technologien: HTML für Struktur, CSS für Darstellung und JavaScript für Verhalten. Diese Vorlesung vermittelt die **Grundlagen** aller drei — nicht als Programmierkurs, sondern als **Architekturwissen** für Web Engineers.

Leitfragen

- Warum ist semantisches HTML mehr als Markup?
- Wie macht JavaScript den Browser zur Laufzeitumgebung?
- Wie erzeugt CSS aus derselben Struktur unterschiedliche Layouts?
- Wie arbeiten HTML, CSS und JavaScript im Browser zusammen?

Diese Vorlesung ist bewusst kein HTML-, CSS- oder JavaScript-Kurs. Es geht nicht darum, Syntax auswendig zu können, sondern darum, die **Architekturentscheidungen** hinter den drei Technologien zu verstehen: warum HTML semantisch sein muss, warum CSS deklarativ ist, warum JavaScript single-threaded läuft. Diese Konzepte bleiben stabil, während die Frameworks alle paar Jahre wechseln. Wenn Sie am Ende der Vorlesung wissen, warum es das Technologie-Dreieck gibt und wo seine Grenzen liegen, haben wir das Lernziel erreicht.

HTML — Struktur und Semantik

Leitfrage: Warum reicht es nicht, eine Web-Seite einfach mit `<div>`-Elementen zusammenzubauen – und was gewinnt man durch semantisches HTML?

HTML ist die älteste der drei Frontend-Sprachen — Tim Berners-Lee hat sie 1991 entworfen, ursprünglich für den Austausch wissenschaftlicher Dokumente am CERN. Bis heute ist HTML ein **Dokumentformat**, kein Programmiersprache. Das ist kein Mangel, sondern eine bewusste Entscheidung: Eine Web-Seite ist zuerst Inhalt, dann Darstellung, dann Verhalten. Wenn Sie diese Reihenfolge umdrehen — also mit JavaScript anfangen und HTML als Beiwerk behandeln — bekommen Sie genau jene Webseiten, die ohne JavaScript leer sind, langsam laden und für Screenreader unbenutzbar bleiben.

Zwei Versionen derselben Seite

Version A

```
<div class="top">
  <div class="logo">Shop</div>
  <div class="links">
    <div><a href="/">Start</a></div>
    <div><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="middle">
  <div class="left">
    <div class="title">Funkkopfhörer</div>
    <div>Kabelloser Kopfhörer mit...</div>
    <div class="price">€ 79,99</div>
  </div>
  <div class="right">
    <div>Auch interessant: ...</div>
  </div>
</div>
<div class="bottom">© 2026 Shop</div>
```

Version B

```
<header>
  <h1>Shop</h1>
  <nav>
    <a href="/">Start</a>
    <a href="/products">Produkte</a>
  </nav>
</header>
<main>
  <article>
    <h2>Funkkopfhörer</h2>
    <p>Kabelloser Kopfhörer mit...</p>
    <p class="price">€ 79,99</p>
  </article>
  <aside>Auch interessant: ...</aside>
</main>
<footer>© 2026 Shop</footer>
```

Aufgabe: Beide Versionen sehen im Browser identisch aus. Was unterscheidet sie — und für wen?

Lassen Sie die Studierenden zwei Minuten das HTML vergleichen, bevor Sie auflösen. Wichtig ist die Pointe: für das Auge eines sehenden Nutzers mit modernem Browser ist *kein* Unterschied erkennbar — beide Versionen rendern visuell identisch. Der Unterschied wird erst dann sichtbar, wenn man die Perspektive wechselt: zu einem Screenreader, einer Suchmaschine, einem Reader-Mode oder einem Wartungsentwickler in zwei Jahren. Diese vier Perspektiven sind das Argument der nächsten Folie.

Was Version B besser macht

Nutzer	Version A (<div>)	Version B (semantisch)
Screenreader	„div, div, div, Link, div, div..." — keine Orientierung	„Navigation mit 2 Links, Hauptinhalt: Artikel ‚Funkkopfhörer‘, Sidebar"
Suchmaschine	Muss raten, was Hauptinhalt ist	<article> + <h2> = klares Inhaltsranking
Entwickler	Muss CSS-Klassen lesen, um Struktur zu verstehen	Code ist selbstdokumentierend
Browser	Kein Reader-Mode, keine sinnvolle Druckansicht	Reader-Mode extrahiert <article>, Druckansicht setzt <main>

Kernprinzip

HTML trennt

- **Struktur** (was ist ein Absatz, eine Überschrift, eine Navigation?)
- von **Darstellung** (wie sieht es aus? → CSS)
- und **Verhalten** (was passiert bei Klick? → JavaScript).

Zwei der vier Perspektiven sind rechtlich relevant: in Deutschland verlangt das Behindertengleichstellungsgesetz (BGG) und die EU-Richtlinie EN 301 549 für öffentliche Einrichtungen barrierefreie Webseiten — semantisches HTML ist die Grundlage dafür. Die SEO-Perspektive ist wirtschaftlich relevant: Google nutzt strukturelle Signale, um Hauptinhalt von Werbung zu trennen, und straft Seiten ab, die nur aus `<div>` bestehen. Die Wartbarkeit-Perspektive ist die, die Studierende oft unterschätzen — bis sie zwei Jahre alten Code ohne Doku übernehmen müssen.

Das Technologie-Dreieck des Webs

Jede Web-Anwendung besteht aus drei Schichten mit klarer Verantwortungsteilung:

Schicht	Technologie	Verantwortung	Leitfrage
Struktur	HTML	Was existiert auf der Seite?	Welche Elemente, welche Bedeutung?
Darstellung	CSS	Wie sieht es aus und wo steht es?	Farbe, Layout, Animation, Responsivität
Verhalten	JavaScript	Was passiert bei Interaktion?	DOM-Änderungen, Server-Kommunikation, Zustand

Warum drei getrennte Technologien?

Dasselbe HTML kann mit **verschiedenem CSS** völlig anders aussehen — dieselbe Shop-Seite als Desktop-Layout, als Mobilansicht, als Druckversion. JavaScript kann **DOM und CSSOM verändern**, aber HTML und CSS können auch **ohne JavaScript** funktionieren: Links navigieren, Formulare senden, Hover-Effekte reagieren.

Diese Schichtung ist kein Zufall — sie ist das Architekturprinzip des Webs.

Diese Trennung wird in der Industrie regelmäßig verletzt — vor allem von Frontend-Frameworks, die HTML, CSS und JS in einer einzigen Komponentendatei bündeln (wir kommen am Ende der Vorlesung darauf zurück). Die Trennung *konzeptionell* zu verstehen, bleibt aber wichtig, weil sie erklärt, warum bestimmte Dinge funktionieren: warum eine Seite ohne JavaScript trotzdem navigierbar ist (HTML allein), warum ein Druck-Stylesheet möglich ist (CSS allein), warum derselbe HTML-Code in mehreren Layouts dargestellt werden kann. Die nächste Folie zeigt das W3C-Standardisierungs-Ökosystem dahinter.

Hinter den Standards: W3C, WHATWG und die Standardisierung des Webs





Die drei Frontend-Technologien sind keine Produkte einer einzelnen Firma. **HTML** wird von der WHATWG (gegründet von Apple, Mozilla, Opera) als *Living Standard* gepflegt; **CSS** und **ARIA** vom W3C;



JavaScript als *ECMAScript* von der ECMA International. Browser implementieren diese Standards — deshalb funktioniert dieselbe Seite in Chrome, Firefox und Safari.

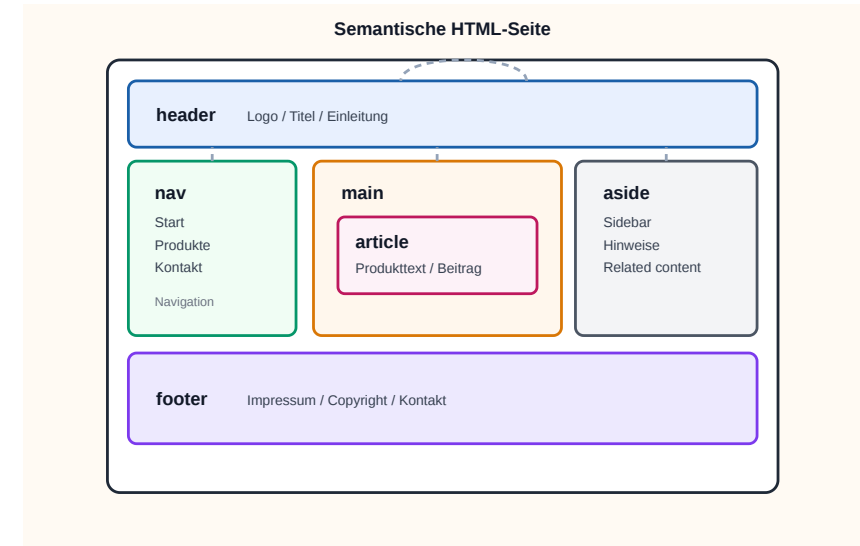
Standardisierung ist der Grund, warum das Web kein iOS und kein Windows ist: keine einzelne Firma kontrolliert es. Das ist auch der Grund, warum das Web so langsam neue Features bekommt — jede Browser-Engine muss zustimmen. Bis 2019 hat das W3C HTML standardisiert, dann hat die WHATWG übernommen, weil das W3C zu langsam war. Heute ist die Arbeitsteilung klar: WHATWG für HTML und DOM, W3C für CSS und ARIA, ECMA für die JavaScript-Sprache. Praktische Konsequenz für Studierende: Wenn Sie wissen wollen, was *standardisiert* ist und was nicht, ist MDN die richtige Anlaufstelle, weil MDN die Browser-Kompatibilität jeder Eigenschaft dokumentiert.



Semantische Elemente in HTML5

HTML5 hat generische Container (<div>,) durch bedeutungstragende Elemente ergänzt:

Element	Bedeutung
<header>	Kopfbereich einer Seite oder Sektion
<nav>	Navigationsbereich
<main>	Hauptinhalt der Seite (einmalig)
<article>	Eigenständiger Inhalt (Blogpost, Produkt)
<section>	Thematischer Abschnitt
<aside>	Ergänzender Inhalt (Sidebar, Hinweis)
<footer>	Fußbereich
<figure> / <figcaption>	Bild mit Beschriftung



Faustregel

Wenn ein Element eine **inhaltliche Bedeutung** hat, gibt es dafür ein semantisches HTML-Element.

<div> ist für Layout-Container ohne eigene Bedeutung.

HTML5 hat 2014 etwa zwei Dutzend semantische Elemente eingeführt, von denen `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>` und `<footer>` die wichtigsten sind. Wichtig ist die Unterscheidung zwischen `<article>` und `<section>`: `<article>` ist eigenständig wiederverwendbar (ein Blogpost, ein Produkt, eine Karte in einem Newsfeed), `<section>` ist nur ein thematischer Abschnitt innerhalb eines größeren Dokuments. `<main>` darf pro Seite nur einmal vorkommen — der Browser-Reader-Mode nutzt es, um den Hauptinhalt zu extrahieren. Für die Studierenden: Im Zweifel sollte man Semantic HTML wählen, auch wenn die visuelle Wirkung null ist — die nicht-sehenden Nutzer der Seite werden es danken.

Semantik kann mehr: Microformats und Microdata

Semantik in HTML endet nicht bei <header> und <article>. Für maschinenlesbare Zusatzinformationen gibt es auch Microformats und Microdata.

```
<a class="h-card" href="/team/max">
  <span class="p-name">Max Mustermann</span>
  
</a>

<article itemscope itemtype="https://schema.org/Person">
  <h1 itemprop="name">Max Mustermann</h1>
  <p itemprop="jobTitle">Data Engineer</p>
</article>
```

- **Microformats** nutzen Klassen wie h-card, p-name, u-photo
- **Microdata** nutzt itemscope, itemtype, itemprop
- Beide ergänzen vorhandenes HTML statt es zu ersetzen

Details: [MDN Microformats](#) und [MDN Microdata](#)

Microformats und Microdata sind keine Pflicht-Inhalte für die Klausur, aber konzeptionell wichtig: Semantik ist nicht binär — `<header>` ist die *grobe* Ebene, Microdata mit `schema.org` die *feine*. Google's Rich-Results-Snippets (Sterne-Bewertungen, Preise, Veranstaltungstermine direkt in der Trefferliste) basieren genau auf diesem strukturierten Markup. JSON-LD im `<script>`-Tag ist heute der gebräuchlichere Mechanismus, weil es das HTML nicht aufbläht — aber das Prinzip bleibt: HTML ist ein Trägermedium für maschinenlesbare Bedeutung.

Wofür braucht man das?

Microformats und Microdata sind nützlich, wenn HTML nicht nur für Menschen lesbar sein soll, sondern auch für Programme, die Inhalte erkennen, zusammenfassen oder weiterverarbeiten.

Typische Nutzung

- **Suchmaschinen:** Produkte, Personen, Organisationen, Events, Bewertungen
- **Aggregatoren und Crawler:** Inhalte automatisch aus Seiten extrahieren
- **Interne Systeme:** strukturierte Daten ohne separates JSON-Format
- **Semantic Web / schema.org:** zusätzliche Bedeutung direkt im HTML

Microformats

- oft für Personen, Kontakte, Termine, Social Links
- leichtgewichtig und direkt lesbar
- Beispiel: `h-card`, `h-event`, `p-name`

Microdata

- häufig für strukturierte Entitäten wie Person, Product, Event
- enger an Vokabulare wie schema.org gebunden
- Beispiel: `itemscope`, `itemtype`, `itemprop`

Was die Nutzung bringt

- bessere maschinelle Erkennbarkeit von Inhalten
- mögliche Rich Results / strukturierte Vorschau in Suchmaschinen
- weniger Parsing-Hacks in externen Tools
- HTML bleibt die primäre Quelle statt nur ein Container für separate Metadaten

Kurz gesagt: Microformats/Microdata ergänzen Semantik dort, wo reine

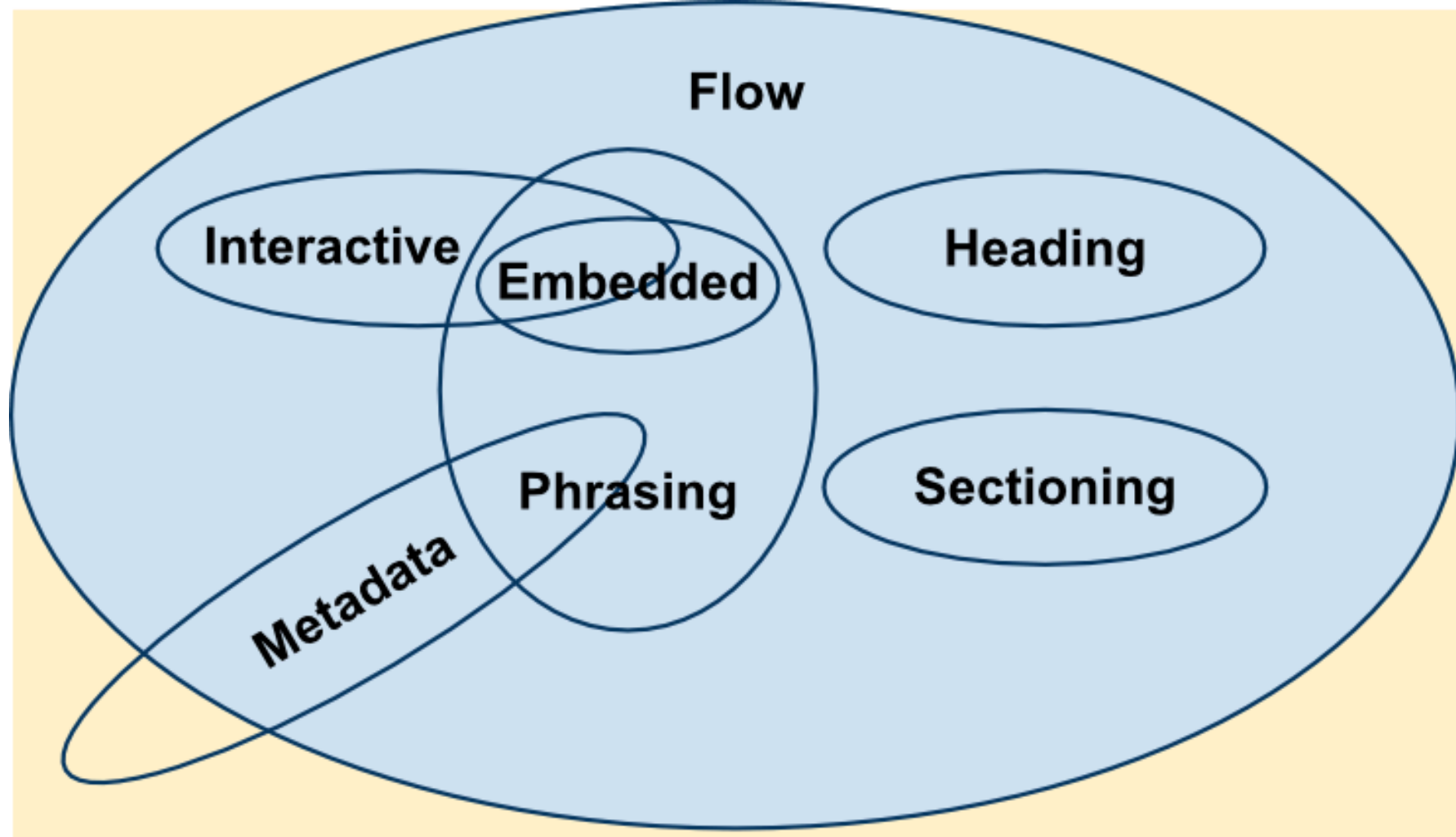


Konkretes Beispiel: Ein Online-Shop, der Produkte mit `schema.org/Product` auszeichnet, bekommt in Google-Trefferlisten oft Sterne, Preis und Verfügbarkeit direkt eingeblendet — was die Click-Through-Rate messbar erhöht. Aggregator-Dienste wie Indeed scrapen Stellenanzeigen automatisch von Firmenseiten, sofern diese `schema.org/JobPosting`-Markup enthalten. Wer also strukturierte Inhalte hat (Produkte, Personen, Termine, Rezensionen), gewinnt durch Microdata reale Sichtbarkeit ohne zusätzlichen Code.

HTML Content Categories

HTML-Elemente gehören zu Kategorien wie Flow, Phrasing, Embedded, Heading, Sectioning, Interactive und Metadata. Diese Kategorien erklären, *wo* ein Element eingesetzt werden darf und *was* es semantisch ist.





Warum das wichtig ist: Content categories sind die formale Schicht hinter HTML-Semantik. Sie helfen beim korrekten Verschachteln von Elementen und beim Verständnis, welche Elemente nur Text tragen, welche eingebettet werden dürfen und welche Metadaten beschreiben.

Die wichtigste praktische Regel: Phrasing-Content (Text und alles, was im Textfluss erlaubt ist — `<span>`, `<a>`, `<em>`) darf in Flow-Content (alles, was im `<body>` erlaubt ist) eingebettet werden, aber nicht umgekehrt. Konkret heißt das: Ein `<div>` in einem `<p>` ist invalides HTML — der Browser bricht den Paragraph implizit ab. Solche stillen Korrekturen führen oft zu schwer auffindbaren Layout-Bugs. Wer die Content Categories kennt, versteht den Validator-Output und vermeidet diese Fallen von vornherein.

Formulare: Die Schnittstelle zum Server

Unser Online-Shop braucht ein Registrierungsformular. HTML bietet dafür eingebaute Validierung:

```
<form action="/api/register" method="POST">
  <label for="email">E-Mail:</label>
  <input type="email" id="email"
        name="email" required>

  <label for="password">Passwort:</label>
  <input type="password" id="password"
        name="password" minlength="8" required>

  <button type="submit">Registrieren</button>
</form>
```

Attribut	Wirkung
required	Feld muss ausgefüllt sein
type="email"	Browser prüft E-Mail-Format
minlength	Mindestlänge des Passworts
pattern	Regex-basierte Validierung

Diskussionsfrage: Der Browser validiert clientseitig. Reicht das? Was passiert, wenn jemand das Formular nicht über den Browser, sondern per `curl` oder `Fetch API` abschickt?

Die richtige Antwort auf die Diskussionsfrage: **Niemals**. Clientseitige Validierung ist ein UX-Feature — sie liefert sofortiges Feedback, ohne den Server zu kontaktieren. Aber sie ist *trivial* zu umgehen: Browser-DevTools, curl, Postman, ein selbstgeschriebenes Skript. Jeder Server muss alle Eingaben unabhängig validieren — das nennt man **Defense in Depth**. Eine gute Faustregel: Clientseitige Validierung ist Höflichkeit gegenüber dem Nutzer, serverseitige Validierung ist Sicherheit. Beide sind nötig. Diese Lektion wiederholt sich in Vorlesung 6 (Datenbanken: SQL Injection) und Vorlesung 7 (Sicherheit: XSS, CSRF).

HTML5 als Plattform

HTML5 hat den Browser von einem Dokumentbetrachter zu einer **Laufzeitumgebung** erweitert:

Fähigkeit	API / Element	Typischer Einsatz
Multimedia	<audio>, <video>	Streaming, Podcasts
Zeichenfläche	<canvas>, WebGL	Spiele, Datenvisualisierung
Lokaler Speicher	localStorage, sessionStorage, IndexedDB	Offline-Daten, Einstellungen
Hintergrundberechnung	Web Workers	CPU-intensive Aufgaben ohne UI-Blockade
Echtzeitkommunikation	WebSockets	Chat, Live-Updates
Geolocation	Geolocation API	Kartenanwendungen
Push-Benachrichtigungen	Notification API + Service Worker	Progressive Web Apps

Architekturentscheidung

Je mehr Fähigkeiten im Browser verfügbar sind, desto mehr Logik **kann** ins Frontend verlagert werden. **Aber sollte sie das?** Ein Online-Shop, der Preise im Client berechnet, riskiert Manipulation. Ein Offline-fähiger Warenkorb dagegen verbessert die User Experience. Die Grenze zwischen Client und Server ist eine bewusste Architekturentscheidung.



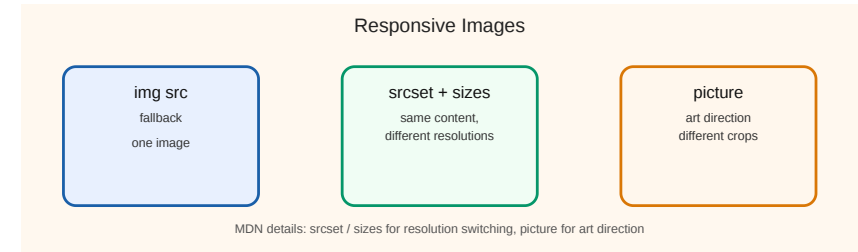
Diese Tabelle ist ein Überblick — keine der Fähigkeiten müssen Sie heute im Detail kennen. Wichtig ist die **Konsequenz**: Der moderne Browser ist faktisch eine Anwendungsplattform, vergleichbar mit Java auf dem Desktop in den 2000ern oder iOS heute. Progressive Web Apps (PWAs) nutzen genau diese Fähigkeiten, um Web-Anwendungen offline-fähig, installierbar und push-fähig zu machen — ohne native App-Stores. Die strategische Frage lautet: Welche Logik gehört in den Browser, welche auf den Server, welche in eine native App? Das ist eine Architekturentscheidung mit Konsequenzen für Sicherheit, Wartung und Reichweite.

Responsive Images

Ein Bild sollte nicht auf jedem Gerät gleich ausgeliefert werden. HTML bietet dafür zwei Werkzeuge:

```
  
  
<picture>  
  <source media="(max-width: 768px)" srcset="hero-mobile.jpg">  
    
</picture>
```

- srcset + sizes = **Resolution Switching**
- <picture> = **Art Direction**
- Details: [MDN Responsive images](https://developer.mozilla.org/en-US/docs/Web/HTML/Element/picture)



Der Unterschied zwischen Resolution Switching und Art Direction wird oft verwechselt. **Resolution Switching** mit `srcset` + `sizes` liefert dasselbe Bild in verschiedenen Auflösungen — der Browser wählt je nach Display-Dichte und Viewport-Breite. **Art Direction** mit `<picture>` und `<source media="...">` liefert *unterschiedliche Bilder* für unterschiedliche Kontexte — etwa ein quadratisches Crop für Mobile statt eines breiten Banner-Crops für Desktop. Praktische Konsequenz: Eine Hero-Aufnahme mit viel Whitespace funktioniert auf Desktop, auf dem Smartphone bleibt davon nichts Erkennbares — Art Direction löst genau dieses Problem.

Kann HTML das allein? Progressive Enhancement

Bevor man CSS oder JavaScript einsetzt, lohnt die Frage: Was kann HTML nativ?

Anforderung	HTML allein?	Erfordert CSS?	Erfordert JS?
Navigation zwischen Seiten	<code><a href></code> — ja	—	—
Formulardaten an Server senden	<code><form action></code> — ja	—	—
E-Mail-Format validieren	<code><input type="email"></code> — ja	—	—
Bild mit Beschriftung	<code><figure></code> + <code><figcaption></code> — ja	—	—
Dreispaltiges Layout	nein	Grid/Flexbox	—
Hover-Effekt auf Button	nein	<code>:hover</code> Pseudo-Klasse	—
Daten vom Server nachladen ohne Seitenreload	nein	nein	<code>fetch()</code>
Drag & Drop im Warenkorb	nein	nein	Event Handling

Progressive Enhancement

Beginne mit funktionierendem HTML. Füge CSS für Darstellung hinzu. Ergänze JavaScript nur wo nötig. Jede Schicht **erweitert**, statt die vorherige zu ersetzen.



Progressive Enhancement ist der Gegenentwurf zu „JavaScript-First-Sites“: Anstatt JavaScript als Voraussetzung anzunehmen und HTML als Beiwerk zu behandeln, wird die Funktionalität *zuerst* in HTML modelliert. Konsequenz: Die Seite funktioniert auf jeder Plattform, mit jedem Browser, ohne JavaScript — sie ist nur weniger komfortabel. Mit JS-Aktivierung kommen Verbesserungen wie Live-Validierung, Inline-Updates, Animationen. Das Gegenmodell — *Graceful Degradation* — geht den umgekehrten Weg: Es startet mit der vollen JS-Erfahrung und versucht, sie für ältere Browser herunterzubrechen. Beides ist legitim, aber Progressive Enhancement ist robuster, weil HTML der stabilere Baseline ist.

Takeaway

HTML definiert die Struktur und Semantik von Web-Inhalten. Der Unterschied zwischen `<div>`-Suppe und semantischem HTML ist nicht kosmetisch — er bestimmt Barrierefreiheit, Auffindbarkeit und Wartbarkeit. Mit HTML5-APIs wird der Browser zur Plattform, was die Frage aufwirft, wie viel Logik ins Frontend gehört. HTML ist deshalb keine reine Markup-Sprache, sondern die Grundlage der Frontend-Architektur.

Wenn Sie aus diesem Modul *eine Sache* mitnehmen, dann diese: HTML ist keine Formatierungssprache. Es ist die Schicht, in der die *Bedeutung* eines Dokuments festgelegt wird — und alle anderen Schichten (CSS, JS, Suchmaschinen, Screenreader, AI-Crawler) verlassen sich darauf. Wer sauberes semantisches HTML schreibt, baut die Grundlage, auf der alle weiteren Optimierungen aufsetzen. Wer Divsuppe schreibt, kämpft danach in jeder anderen Schicht gegen seine eigene Struktur.

JavaScript — Die Sprache des Webs

Leitfrage: JavaScript ist die einzige Programmiersprache, die nativ in jedem Browser läuft. Was muss man über sie wissen, um Web-Systeme architektonisch zu verstehen?

JavaScript ist historisch ein Unfall — Brendan Eich hat es 1995 in zehn Tagen entworfen, um statische HTML-Seiten dynamisch zu machen. Aus diesem Provisorium ist die meistbenutzte Programmiersprache der Welt geworden. Architektonisch wichtig ist nicht die Syntax, sondern das **Ausführungsmodell**: single-threaded, ereignisbasiert, asynchron. Wer das versteht, versteht warum SPAs funktionieren und warum Node.js auf dem Server existiert.

Was gibt dieser Code aus?

```
console.log("1: Start");

setTimeout(() => console.log("2: Timeout"), 0);

fetch('/api/products/42')
  .then(response => response.json())
  .then(data => console.log("3: Daten geladen"));

console.log("4: Ende");
```

Aufgabe: In welcher Reihenfolge erscheinen die vier Zeilen in der Konsole? Begründen Sie.

Lassen Sie die Studierenden zwei Minuten überlegen, bevor Sie auflösen. Die intuitive (falsche) Erwartung ist meistens, dass `setTimeout(..., 0)` „sofort“ ausgeführt wird oder dass `fetch` synchron blockiert. Die richtige Antwort erfordert das Verständnis, dass JavaScript single-threaded ist und alle asynchronen Operationen erst nach dem aktuellen Skript abgearbeitet werden. Wer das hier richtig vorhersagt, hat das Event-Loop-Modell verstanden — und kann die meisten Bug-Berichte über „die Daten kommen zu spät an“ einordnen.

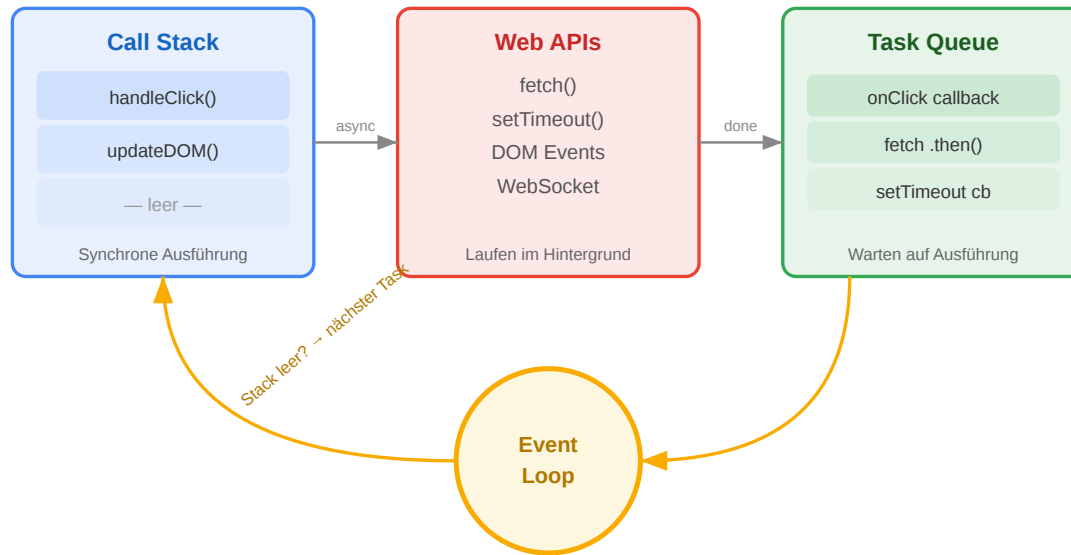
Auflösung

1: Start → 4: Ende → 2: Timeout → 3: Daten geladen

JavaScript ist **single-threaded** — aber `setTimeout` und `fetch` laufen im Hintergrund. Der aktuelle Code wird zuerst vollständig ausgeführt, dann kommen die Callbacks. Das ist das fundamentale Ausführungsmodell, das Web-Anwendungen performant macht.

Wichtig zu betonen: `setTimeout(..., 0)` bedeutet *nicht* „sofort“, sondern „so früh wie möglich, nachdem der aktuelle Stack leer ist“. Selbst mit Verzögerung 0 wird der Callback in die Task Queue gestellt und dort vom Event Loop abgearbeitet. Das gleiche gilt für Promises (`.then`) und `fetch` — sie kommen aus der Microtask Queue, die vor der Task Queue abgearbeitet wird, weshalb `Promise.resolve().then(...)` in der Praxis vor einem `setTimeout(..., 0)` läuft. Für die Vorlesung reicht das vereinfachte Modell „aktueller Code zuerst, alles andere danach“.

Der Event Loop: Warum das Web nicht blockiert



Wie es funktioniert

1. **Call Stack:** Führt den aktuellen Code synchron aus
2. **Web APIs:** Browser übernimmt Netzwerk, Timer, Events im Hintergrund
3. **Task Queue:** Fertige Callbacks warten hier
4. **Event Loop:** Wenn der Call Stack leer ist, holt er den nächsten Callback

Warum das architektonisch wichtig ist

- Ein **einzig**er Thread für UI und Logik → blockierende Operationen frieren die Seite ein
- Deshalb ist **alles I/O asynchron** (Netzwerk, Timer, File API)
- Node.js nutzt dasselbe Modell serverseitig → ein Thread bedient tausende Verbindungen

Die wichtige Konsequenz für die Praxis: Wenn Sie in JavaScript eine Schleife haben, die 500ms läuft, friert die Seite für 500ms komplett ein — keine Klicks, keine Animationen, keine Eingaben. Der Browser kann erst wieder reagieren, wenn der Call Stack leer ist. Lange Berechnungen müssen deshalb entweder in **Web Workers** ausgelagert werden (echter zweiter Thread) oder in kleine Häppchen aufgeteilt und mit `setTimeout` oder `requestIdleCallback` über die Zeit verteilt werden. Dass Node.js dasselbe Modell serverseitig verwendet, ist kein Zufall: Webserver verbringen die meiste Zeit mit Warten auf I/O — ein einziger Thread mit Event Loop kann tausende Verbindungen bedienen, weil er nie blockiert.

Der DOM: HTML als programmierbare Struktur

Der Browser parst HTML und erzeugt den **Document Object Model (DOM)** — einen Baum, den JavaScript lesen und verändern kann:

```
document
├── html
│   ├── head
│   └── body
│       ├── h2 "Funkkopfhörer"
│       ├── p  "€ 79,99"
│       └── button "In den Warenkorb"
```

Produktseite unseres Online-Shops

```
// Preis aus dem DOM lesen
const price = document
  .querySelector('.price')
  .textContent;

// Badge aktualisieren
const badge = document
  .querySelector('.cart-badge');
badge.textContent = '3';

// Neues Element erzeugen
const msg = document.createElement('p');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);
```

Kernidee

Der DOM ist die

Brücke zwischen HTML und JavaScript

. Jede Änderung am DOM wird sofort im Browser sichtbar.

Der DOM ist nicht das HTML — der Browser parst HTML und erzeugt daraus den DOM-Baum. Diese Unterscheidung wird wichtig, sobald JavaScript ins Spiel kommt: Sie können im DOM Knoten erzeugen, die nie im HTML-Quelltext standen. Wenn Sie im Browser „View Source“ wählen, sehen Sie das ursprüngliche HTML; wenn Sie in den DevTools den Inspector öffnen, sehen Sie den aktuellen DOM-Stand. Bei einer modernen Single-Page-App können diese beiden komplett unterschiedlich aussehen — was ein Problem für SEO und View-Source-Crawler ist (Stichwort Server-Side Rendering, das wir am Ende der Vorlesung streifen).

JavaScript als Brücke zwischen HTML und CSS

JavaScript ist die **einzige** Frontend-Technologie, die sowohl den DOM (HTML) als auch das CSSOM (CSS) verändern kann. Das macht JS zum **Orchestrator**:

CSSOM bedeutet *CSS Object Model*: der interne Baum der CSS-Regeln, den der Browser aus Stylesheets und Inline-Styles aufbaut. Wenn JavaScript Klassen, Inline-Styles oder Custom Properties ändert, beeinflusst es damit nicht nur HTML, sondern auch die berechnete Darstellung.

```
// JS → HTML: DOM-Struktur ändern
const msg = document.createElement('div');
msg.textContent = 'Hinzugefügt!';
document.body.appendChild(msg);

// JS → CSS: Klassen steuern Darstellung
button.classList.add('loading');      // CSS-Transition startet
button.classList.remove('loading');   // CSS-Transition endet

// JS → CSS: Custom Properties dynamisch setzen
document.documentElement.style.setProperty('--cart-count', '3');
```

Die Richtung der Abhängigkeit

```
HTML  ←-- JavaScript  --→  CSS
(DOM)   liest & ändert  (CSSOM)
```



- **HTML → JS:** HTML existiert ohne JS (Links, Formulare funktionieren). JS **erweitert** HTML.
- **CSS → JS:** CSS reagiert auf Zustände (:hover, :checked) ohne JS. JS **triggert** CSS-Übergänge über Klassen.
- **JS → beide:** Nur JS kann DOM und CSSOM gleichzeitig manipulieren — deshalb ist JS die Verhaltensschicht.

Konsequenz: Wann immer ein visueller Effekt auch per CSS lösbar ist (Transition, Animation, Hover), sollte CSS es übernehmen — JS steuert nur den **Auslöser** (Klasse setzen), nicht die **Animation** selbst.

Diese Aufteilung ist nicht nur Stil — sie ist auch Performance. CSS-Transitions und -Animations laufen im Compositor-Thread des Browsers, also *außerhalb* des JavaScript-Hauptthreads. Eine in CSS animierte Karte bleibt flüssig, auch wenn JavaScript gerade beschäftigt ist. Eine in JavaScript animierte Karte (mit `requestAnimationFrame` und manuellem Setzen von `style.left`) ruckelt, sobald JS andere Arbeit hat. Diese architektonische Trennung „JS triggert, CSS animiert“ ist die Grundlage für 60-FPS-Webseiten.

Events: Auf Nutzeraktionen reagieren

JavaScript arbeitet **ereignisgesteuert**: Code wird ausgeführt, wenn etwas passiert. In unserem Online-Shop:

```
const button = document.querySelector('#add-to-cart');

button.addEventListener('click', async (event) => {
  // 1. Sofortiges visuelles Feedback
  button.textContent = 'Wird hinzugefügt...';
  button.disabled = true;

  // 2. Server kontaktieren (asynchron)
  const response = await fetch('/api/cart', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ productId: 42, quantity: 1 })
  });

  // 3. UI aktualisieren basierend auf Serverantwort
  if (response.status === 201) {
    button.textContent = '✓ Im Warenkorb';
    updateCartBadge();
  }
});
```

Diskussionsfrage: Der Button wird sofort deaktiviert und der Text geändert, **bevor** die Serverantwort eintrifft. Warum macht man das? Was passiert, wenn die Netzwerkverbindung abbricht?

Antwort auf die Diskussionsfrage: Das Muster heißt **optimistisches UI-Update**. Der Nutzer bekommt sofort Feedback, dass sein Klick angekommen ist — auch wenn der Server noch nicht geantwortet hat. Das fühlt sich schneller an, weil nicht 200ms Netzwerklatenz wahrgenommen werden müssen. Risiko: Wenn der Server fehlschlägt, muss das UI **zurückgerollt** werden — der Button-Text wird zurückgesetzt, eine Fehlermeldung erscheint. Das ist die schwierigere Hälfte des Patterns und der Grund, warum Frameworks wie React Query oder SWR einen Großteil ihrer API genau dieser Frage widmen: Wie macht man Optimistic Updates korrekt rückgängig?

Fetch API: Die Client-Seite von HTTP

Die Fetch API ist die Brücke zwischen Frontend (Vorlesung 03) und Protokoll (Vorlesung 04):

```
// GET: Produktliste laden
const response = await fetch('/api/products?category=electronics&sort=-price');
const products = await response.json();

// POST: Bestellung aufgeben
const result = await fetch('/api/orders', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ productId: 42, quantity: 1 })
});
if (result.status === 201) {
  const order = await result.json();
  window.location.href = `/orders/${order.id}`; // Weiterleitung
}
```

Fetch-Konzept	HTTP-Gegenstück
method: 'POST'	HTTP-Methode — definiert die Absicht
headers	HTTP Request Headers — Metadaten
response.status	HTTP-Statuscode — Ergebnis der Operation
response.json()	Body parsen — die Repräsentation der Ressource

Die Fetch API ist die moderne Schnittstelle zwischen JavaScript und HTTP — sie hat den älteren XMLHttpRequest (XHR) abgelöst. Konzeptionell wichtig: `fetch()` gibt ein **Promise** zurück, das sich auflöst, sobald die *Header* der Antwort eingetroffen sind — nicht der vollständige Body. `response.json()` ist ein zweiter Schritt, der den Body parst und ebenfalls ein Promise zurückgibt. Das mag umständlich wirken, ermöglicht aber Streaming: Bei einer großen Antwort können Sie schon mit den Headern arbeiten (z.B. Status oder Content-Type prüfen), bevor der gesamte Body geladen ist. Diese Tabelle zeigt explizit, wie 1:1 die Fetch-API-Felder auf HTTP-Konzepte abbilden — eine Brücke zur nächsten Vorlesung.

Was kann schiefgehen?

Aufgabe: Dieser Code funktioniert im Normalfall. Welche Probleme können auftreten — und wie sollte man sie behandeln?

```
const response = await fetch('/api/products/42');  
const product = await response.json();  
document.querySelector('.name').textContent = product.name;  
document.querySelector('.price').textContent = product.price + ' €';
```

Speaker notes

Lassen Sie die Studierenden zwei Minuten überlegen, bevor Sie zur nächsten Folie weitergehen. Mindestens fünf Probleme sollten gefunden werden: Server unerreichbar, 404, langsame Verbindung, kein JSON, fehlendes DOM-Element. Wer alle fünf findet, hat ein gutes Gespür für Fehlerquellen entwickelt — die meisten Studierenden finden zwei oder drei. Diese Lücke zwischen „funktioniert im Glücksfall" und „funktioniert robust" ist der Kern des Unterschieds zwischen Hobby- und Produktivcode.



Probleme und Lösungen

Problem	Was passiert	Lösung
Server nicht erreichbar	fetch wirft Exception	try/catch + Fehlermeldung anzeigen
404 Not Found	response.ok ist false, .json() liefert Fehler-Body	Status prüfen vor .json()
Langsame Verbindung	UI zeigt alte/keine Daten	Loading-Indicator anzeigen
Response ist kein JSON	.json() wirft Exception	Content-Type prüfen
DOM-Element existiert nicht	querySelector liefert null → Fehler	Null-Check oder optionale Verkettung (?.)

Robuste Fehlerbehandlung ist keine Kür — sie ist der Unterschied zwischen einem Prototyp und einem Produktivsystem.

Eine subtile Falle bei Fetch: `fetch()` wirft *nur* dann eine Exception, wenn das Netzwerk komplett versagt (kein DNS, keine Verbindung). HTTP-Statuscodes wie 404 oder 500 gelten technisch als „erfolgreich übertragen“ — `fetch` liefert ein gültiges Response-Objekt, und `response.ok` ist `false`. Wer das nicht prüft, behandelt Fehlerseiten wie gültige Daten und versucht, HTML-Errorpages als JSON zu parsen. Das ist eine der häufigsten Anfänger-Fallen mit der Fetch API. Robuste Patterns prüfen immer `if (!response.ok) throw ...` vor dem `.json()`-Aufruf.

Warum Frameworks existieren: Das Skalierungsproblem

Unser Warenkorb-Button funktioniert mit `querySelector` und `addEventListener`. Was passiert, wenn die Anwendung wächst?

```
// 5 Zeilen für einen Button – übersichtlich
button.addEventListener('click', () => { badge.textContent = '3'; });

// Aber: 50 Produkte, Filter, Sortierung, Pagination, Warenkorb-Sidebar...
// → Hunderte querySelector-Aufrufe, manuelles DOM-Tracking, Zustandschaos
```

Das Problem imperativer DOM-Manipulation

Komplexität	Vanilla JS	Framework (React/Vue)
Ein Button	Einfach, direkt	Overhead — zu viel Abstraktion
10 interaktive Elemente	Machbar, wird unübersichtlich	Komponenten helfen
Komplexe SPA (Shop, Dashboard)	Zustand und DOM divergieren	Deklarativ: UI = f(state)

Der konzeptionelle Unterschied

- **Imperativ** (Vanilla JS): „Finde Element X, ändere Property Y, füge Z ein“
- **Deklarativ** (React/Vue): „So soll die UI aussehen, wenn der Zustand S ist“ — Framework berechnet die DOM-Änderungen

Frameworks lösen nicht ein Syntax-Problem, sondern ein **Zustandsmanagement-Problem**: Wie bleibt die UI konsistent, wenn sich Daten ändern?

Die zentrale Erkenntnis: Frameworks wie React sind keine syntaktischen Verschönerungen, sondern eine **konzeptionelle Verschiebung**. Imperatives DOM („greife Element, ändere Property“) wird durch deklarative UI ersetzt („so soll es aussehen, wenn der Zustand X ist“). Das Framework berechnet automatisch die minimalen DOM-Änderungen — das sogenannte „Reconciliation“ oder „Virtual DOM Diffing“ bei React. Für kleine Anwendungen ist das Overkill, weil 50 Zeilen Vanilla JS reichen. Sobald aber Zustand an mehreren Stellen synchron gehalten werden muss (Warenkorb-Badge, Sidebar, Modal, Hauptanzeige), wird imperatives DOM-Tracking unwartbar und der Framework-Overhead lohnt sich.

JavaScript-Ökosystem: Überblick

Bereich	Technologie	Rolle
Browser-Frameworks	React, Vue.js, Angular, Svelte	Deklarative, komponentenbasierte UI
Server-Runtime	Node.js, Deno, Bun	Dasselbe Event-Loop-Modell serverseitig
Full-Stack Frameworks	Next.js, Nuxt.js, SvelteKit	Server-Rendering + Client-Interaktion
TypeScript	Typisierte Obermenge von JS	Statische Typprüfung für große Codebases
Build-Tools	Vite, webpack, esbuild	Bündelung, Transpilation, Optimierung

Die Wahl des Frameworks ist eine **Architekturentscheidung** — sie bestimmt, wie Zustand verwaltet, Code organisiert und die Anwendung deployt wird.

Diese Tabelle ist bewusst nicht vollständig — das JavaScript-Ökosystem ist riesig und ändert sich schnell. Wichtig sind die *Kategorien*: Browser-Frameworks (UI), Server-Runtimes (Node.js und Konkurrenz), Full-Stack-Frameworks (Server + Client kombiniert), TypeScript als Typsystem und Build-Tools für die Auslieferung. Wer diese fünf Kategorien kennt, kann jede neue Technologie sofort einordnen. Die Wahl ist nie technisch neutral: Sie bestimmt, wie das Team arbeitet, welche Bibliotheken zur Verfügung stehen, wie deployment funktioniert und wie schwer es ist, neue Entwickler einzuarbeiten.

Takeaway

JavaScript ist die **Verhaltensschicht** des Webs und zugleich der Orchestrator: Es ist die einzige Technologie, die sowohl DOM (HTML) als auch CSSOM (CSS) verändern kann. Das Ausführungsmodell — single-threaded mit Event Loop — ermöglicht responsive UIs trotz Netzwerk-Latenz. Die Fetch API verbindet Frontend mit Backend. Für Web Engineering sind drei Konzepte entscheidend: das **Event-Loop-Modell**, die **Abhängigkeitsrichtung** (JS erweitert HTML/CSS, nicht umgekehrt) und die **Grenze zu Frameworks** (ab welcher Komplexität lohnt deklarative UI).

Speaker notes

Wenn Sie aus diesem Modul *eine Sache* mitnehmen sollen, dann das Event-Loop-Modell — es erklärt, warum JavaScript single-threaded sein kann und trotzdem nicht blockiert, warum `async/await` syntaktischer Zucker ist und nicht echte Parallelität, warum lange Berechnungen die UI einfrieren und warum Node.js auf demselben Server tausende Verbindungen bedienen kann. Alles andere — DOM, Events, Fetch, Frameworks — baut auf diesem einen Modell auf.



CSS — Layout und Responsive Design

Leitfrage: Warum braucht das Web eine eigene Sprache für Darstellung — und wo endet die Verantwortung von CSS, wo beginnt die von JavaScript?

CSS wirkt auf den ersten Blick trivial — Farben, Schriftgrößen, ein bisschen Layout. Tatsächlich ist CSS die anspruchsvollste der drei Frontend-Sprachen, weil sie deklarativ ist: Sie beschreiben *Constraints*, der Browser löst sie auf. Die meisten Bugs in Frontend-Code sind keine JavaScript-Fehler, sondern unverstandene CSS-Spezifität, Cascade oder Layout-Wechselwirkungen. In diesem Modul geht es weniger um Syntax als um die mentalen Modelle, die hinter CSS stehen.

Welche Farbe hat der Text?

```
<p id="info" class="highlight">Willkommen im Shop</p>
```

```
p      { color: blue; }  
.highlight { color: green; }  
#info   { color: red; }  
p.highlight { color: orange; }
```

Aufgabe: Der Paragraph matcht alle vier Selektoren. Welche Farbe wird angezeigt — und warum?

Lassen Sie die Studierenden überlegen, bevor Sie auflösen. Die meisten werden intuitiv denken „die letzte Regel gewinnt“ — das ist falsch. CSS ist nicht zeilenorientiert, sondern spezifitätsorientiert. Die Reihenfolge im Stylesheet ist nur der Tiebreaker, wenn die Spezifität identisch ist. Diese falsche Intuition ist die Wurzel der meisten CSS-Bugs in der Praxis: „Ich habe die Regel doch ganz unten geschrieben, warum greift sie nicht?“

Auflösung: Spezifität entscheidet

Selektor	Spezifität (ID, Klasse, Element)	Farbe
p	(0, 0, 1)	blue
.highlight	(0, 1, 0)	green
p.highlight	(0, 1, 1)	orange
#info	(1, 0, 0)	red

ID schlägt Klasse schlägt Element. In großen Projekten führen Spezifitätskonflikte zu unerwartetem Verhalten — deshalb setzen moderne Projekte auf Konventionen (BEM) oder komponentenbezogenes CSS (Scoped Styles).

Die Spezifität wird als Tripel (ID, Klasse, Element) berechnet und lexikografisch verglichen: $(1, 0, 0)$ schlägt $(0, 99, 99)$. Inline-Styles (`style="..."`) haben die noch höhere Spezifität $(1, 0, 0, 0)$. `!important` überschreibt alles — und ist deshalb verpönt, weil es die Kaskade außer Kraft setzt und spätere Überschreibungen unmöglich macht. BEM (Block-Element-Modifier) ist eine Namenskonvention, die Spezifitätskonflikte vermeidet, indem jede Komponente mit nur einer Klasse gestylt wird. Scoped Styles in Vue oder CSS Modules in React lösen dasselbe Problem automatisch, indem sie Klassennamen pro Komponente eindeutig machen.

Warum existiert CSS als eigene Technologie?

In den 1990ern wurde Darstellung direkt in HTML geschrieben: `<font color="red" size="5">`. Das führte zu einem fundamentalen Problem:

Problem	Ohne CSS	Mit CSS
Redesign	Jede HTML-Seite einzeln ändern	Ein Stylesheet ändern → alle Seiten betroffen
Verschiedene Geräte	Eine Version für alle	Verschiedene Stylesheets: Screen, Print, Mobile
Barrierefreiheit	Visuelles Layout = Dokumentstruktur	Screenreader ignoriert CSS, liest Struktur
Teamarbeit	Designer ändert HTML	Designer ändert CSS, Entwickler ändert HTML

Das Kernkonzept: Ein HTML, viele Darstellungen

Dieselbe Shop-Seite kann mit **verschiedenem CSS** als Desktop-Layout, als mobile Ansicht, als Druckversion oder im Dark Mode erscheinen — ohne das HTML zu ändern. Das ist kein Feature, sondern der **Grund, warum CSS als separate Sprache existiert**.

Normal Flow zuerst



- HTML-Elemente folgen zunächst dem **normal flow**
- **Block-Elemente** beginnen auf einer neuen Zeile und nehmen typischerweise die volle verfügbare Breite ein
- **Inline-Elemente** bleiben in derselben Zeile und nehmen nur so viel Platz ein wie ihr Inhalt braucht
- Erst danach greifen `display`, `position`, Flexbox oder Grid als bewusste Abweichungen vom Standard

So sieht das aus

```
<p>Absatz <span>Inline-Text</span> nach dem Wort.</p>  
<div>Dieses Block-Element steht in einer eigenen Zeile.</div>
```

Im Browser: Der Absatztext läuft in einer Zeile, das `span` bleibt im Textfluss, und das `div` startet darunter auf einer neuen Zeile.

CSS ist deklarativ, nicht imperativ

CSS sagt nicht „zeichne ein Rechteck bei Pixel 200,300“. CSS sagt „dieses Element soll 80% breit sein, zentriert, mit 1rem Abstand“. Der **Browser** berechnet die konkreten Pixel. Das ist fundamental anders als Canvas/JavaScript-Zeichnen und der Grund, warum CSS-Layouts automatisch auf verschiedene Bildschirmgrößen reagieren können.

Der historische Kontext ist wichtig: Vor CSS wurden Webseiten mit `<font>`, `<center>`, `<table>` für Layout und inline-Attributen wie `bgcolor` gestylt. Das hatte drei Probleme: Wartung (jede Seite einzeln ändern), Gerätevielfalt (eine Version für alle) und Barrierefreiheit (Layout-Tabellen verwirren Screenreader). CSS wurde 1996 spezifiziert, um diese Probleme zu lösen — indem Darstellung von Struktur getrennt wurde. Die Trennung ist so fundamental, dass sie bis heute das Grundprinzip des Webs ist — selbst Komponenten-Frameworks, die HTML/CSS/JS bündeln, halten die Schichten konzeptionell getrennt. Der Normal Flow ist die Default-Anordnung: block elements stapeln sich vertikal, inline elements fließen horizontal. Erst `display`, `position`, Flexbox oder Grid verändern das bewusst.

CSS in Kurzform

CSS besteht aus **Selektoren** und **Deklarationen**:

```
p.highlight {  
  color: orange;  
  font-weight: 600;  
}
```

- **Selektor:** `p.highlight` sagt, *welche* Elemente gemeint sind
- **Deklaration:** `color: orange;` sagt, *wie* sie aussehen sollen
- **Regelblock:** alles zwischen `{ ... }`

Was CSS auflöst

- **Kaskade:** mehrere Regeln können denselben Stil setzen
- **Spezifität:** manche Selektoren gewinnen bei Konflikten
- **Vererbung:** manche Eigenschaften wandern von Eltern zu Kindern

Merksatz: CSS beschreibt Stil als Regeln, nicht als Pixelbefehle.

Pseudo-Klassen



Pseudo-Klassen sind Selektoren für **Zustände** oder **Positionen** eines Elements. Sie beginnen mit `:` und hängen an einen normalen Selektor an.

```
a:hover {  
  text-decoration: underline;  
}  
  
input:focus {  
  border-color: #1a5fa8;  
}
```

- `:hover` trifft ein Element, wenn der Mauszeiger darüber liegt
- `:focus` trifft ein Element, wenn es per Tastatur oder Klick aktiv ist
- Pseudo-Klassen beschreiben also nicht *was* ein Element ist, sondern *in welchem Zustand* es gerade ist

Zwei typische Beispiele:

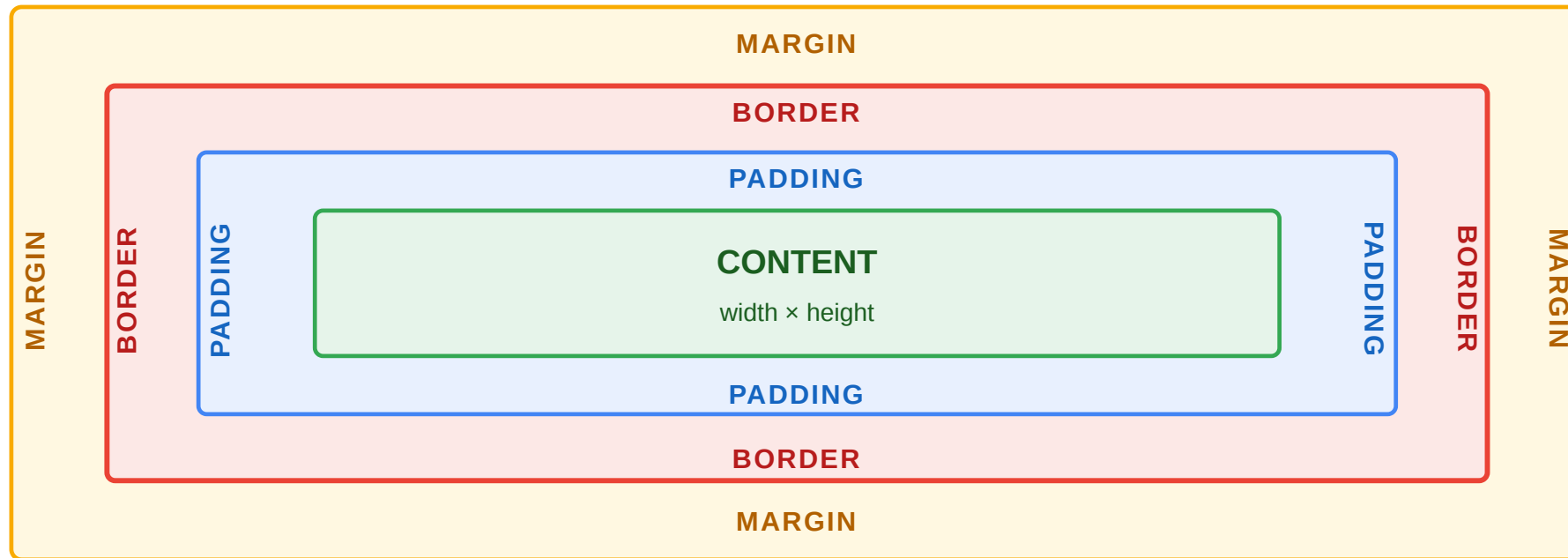
- `button:hover` für Hover-Effekte
- `input:focus` für fokussierte Formularfelder

Pseudo-Klassen sind der Grund, warum CSS *reaktiv* ohne JavaScript sein kann. `:hover` reagiert auf Maus-Position, `:focus` auf Tastatur-Navigation, `:checked` auf Checkbox-Zustand, `:valid/:invalid` auf Formular-Validität. Die Kaskade wird regelmäßig mit dem „Kaskadierungsprinzip“ erklärt: Von mehreren gültigen Regeln gewinnt die spezifischste, bei gleicher Spezifität die spätere. Diese Sortierung ist deterministisch — aber das Tripel aus Ursprung, Spezifität und Reihenfolge ist für Anfänger die häufigste Verwirrungsquelle. Die Vererbung ist ein zweites Konzept: manche Eigenschaften (`color`, `font`) wandern von Eltern zu Kindern, andere (`border`, `padding`) nicht.

Das Box-Modell

Jedes HTML-Element ist eine Box: **Content** → **Padding** → **Border** → **Margin**

Die häufigste Fehlerquelle: `width: 200px` bedeutet standardmäßig nur den Content — Padding und Border kommen **obendrauf**. Erst `box-sizing: border-box` macht `width` zum Gesamtmaß.



Das Box-Modell ist das erste wirklich nicht-intuitive Detail von CSS: `width: 200px + padding: 20px + border: 2px` ergibt im Default-Box-Modell ein Element mit 244px tatsächlicher Breite — und zerschießt jedes Raster-Layout. Deshalb beginnt fast jedes moderne Stylesheet mit `* { box-sizing: border-box; }` als Reset. Dann entspricht `width: 200px` der tatsächlichen Breite, und Padding und Border werden innen abgezogen. Das ist eine der wenigen Stellen, an denen CSS einen schlechten Default hat — historisch gewachsen und nicht mehr änderbar, weil es unzählige bestehende Webseiten brechen würde.

Was CSS ohne JavaScript kann

Bevor man JavaScript für visuelle Effekte einsetzt, lohnt die Frage: Kann CSS das allein?

Anforderung	CSS-Lösung	JS nötig?
Hover-Effekt auf Produktkarte	:hover Pseudo-Klasse	nein
Warenkorb-Button-Animation	transition + :active	nein
Dark Mode	@media (prefers-color-scheme: dark)	nein
Formularfeld-Validierung visuell	:valid, :invalid, :focus	nein
Akkordeon (auf/zuklappen)	<details> + CSS	nein
Smooth Scroll	scroll-behavior: smooth	nein
Drag & Drop sortieren	—	ja
Daten vom Server nachladen	—	ja

Die Grenze

CSS reagiert auf **Zustände** (:hover, :checked, :focus, @media) und **animiert Übergänge**. Sobald **neuer Zustand erzeugt** werden muss (Daten laden, DOM-Elemente erstellen, Server kontaktieren), braucht man JavaScript. JavaScript sollte den **Auslöser** steuern (Klasse setzen), CSS die **Darstellung** (Transition/Animation).

Speaker notes

Diese Tabelle ist wichtig für die Architektur-Denke. CSS kann *sehr viel* mehr, als die meisten Anfänger annehmen: Akkordeons mit <details>, Dropdown-Menüs mit :focus-within, Formular-Validierung mit :valid/:invalid, ganze Themes mit Custom Properties und @media (prefers-color-scheme). Das spart JavaScript — und damit Ladezeit, Wartungsaufwand, Komplexität. Die Faustregel: Prüfen Sie immer zuerst, ob CSS das allein kann. Wenn ja, nehmen Sie CSS. Wenn nein, überlegen Sie, ob der Aufwand in JavaScript wirklich gerechtfertigt ist.



Zentrale Layoutmodelle

Jedes Layoutmodell löst ein **anderes Verteilungsproblem**:

Modell	Konzept	Denkmodell	Typischer Einsatz
Normal Flow	Default-Verhalten von block und inline	Dokument lesen	Textseiten, einfache Layouts
Flexbox	Platz in einer Richtung verteilen	Regal: Elemente in einer Zeile oder Spalte	Navigation, Kartenreihe, Toolbar
Grid	2D-Raster mit Zeilen und Spalten	Tischdecke mit Koordinaten	Seitenlayout, Dashboard, Galerie
Positioning	Element aus dem Flow herausnehmen	Post-it auf eine Seite kleben	Overlay, Tooltip, Sticky Header

Die konzeptionelle Unterscheidung

- **Flow** beantwortet: *Was passiert ohne Layoutanweisung?*
- **Flexbox** beantwortet: *Wie verteilen wir Elemente entlang einer Achse?*
- **Grid** beantwortet: *Wie legen wir ein 2D-Raster fest?*
- **Positioning** beantwortet: *Wann soll ein Element den Flow verlassen?*

Wichtig: Flexbox und Grid sind keine konkurrierenden Varianten desselben Problems, sondern zwei verschiedene Antworten auf zwei verschiedene räumliche Fragen.

Flexbox vs. Grid — Produktliste des Online-Shops

```
/* Flexbox: Produkte verteilen sich in einer Reihe, brechen bei Platzmangel um */  
.product-list { display: flex; flex-wrap: wrap; gap: 1rem; }  
.product-card { flex: 1 1 300px; }  
  
/* Grid: Festes 3-Spalten-Raster, alle Karten gleich groß */  
.product-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }
```

Faustregel: Flexbox wenn der **Inhalt** das Layout bestimmt (Elemente verteilen sich). Grid wenn das **Layout** den Inhalt bestimmt (Raster vorgeben).

Vor Flexbox (2012) und Grid (2017) war CSS-Layout ein Albtraum: vertikal zentrieren erforderte Tricks mit `display: table-cell`, Spaltenlayouts wurden mit `float` gebaut, das Clearen von Floats brauchte Hacks wie `clearfix`. Flexbox und Grid haben CSS-Layout fundamental vereinfacht — was früher 20 Zeilen JavaScript-Workaround brauchte, ist heute drei Zeilen CSS. Für Studierende wichtig: Die alten Techniken (Floats, Positioning, Tables) tauchen in Legacy-Code noch auf, aber für neuen Code sind Flexbox und Grid fast immer die richtige Wahl. Die Entscheidung zwischen beiden folgt der oben genannten Faustregel.

Grid: das 2D-Raster

Grid ist CSS für Fälle, in denen wir **Zeilen und Spalten zugleich** kontrollieren wollen.

Konzeptionell

- Flexbox verteilt Platz in **einer** Richtung
- Grid legt ein **Raster** aus Zeilen und Spalten fest
- Elemente werden dann in dieses Raster eingeordnet
- Ideal für Seitenlayout, Dashboards und komplexe Formulare

Ein einfaches Beispiel

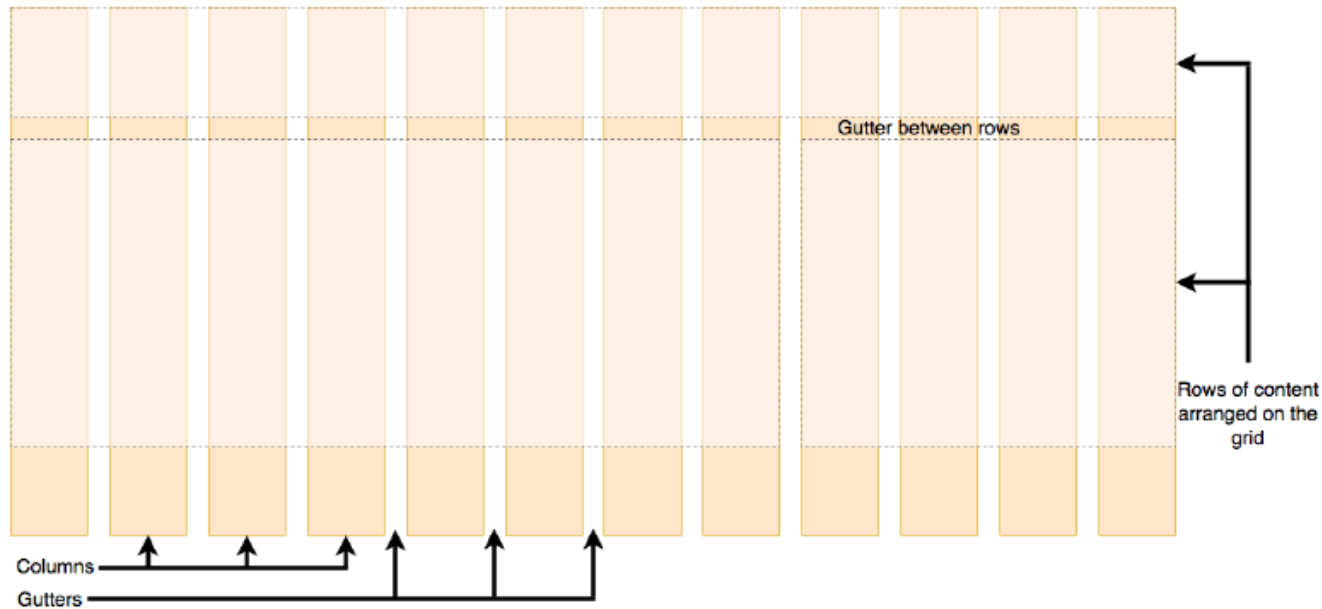
```
.shop-layout {  
  display: grid;  
  grid-template-columns: 250px 1fr 200px;  
  grid-template-rows: auto 1fr auto;  
  grid-template-areas:  
    "header header header"  
    "nav   main   aside"  
    "footer footer footer";  
  gap: 1rem;  
  min-height: 100vh;  
}
```

Merksatz: Grid organisiert Raum als Tabelle mit benannten Bereichen.



Fluid Grids

CSS Grid Layout unterteilt eine Seite in Regionen und deren geometrische Beziehung zueinander



Defining a grid

Defining the columns

`container {`

```
.container {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  grid-auto-rows: minmax(100px, auto);  
  gap: 20px;  
}
```

- fr is fraction of available space
- Rows are added automatically
- Define row sizes (minimum / maximum)

The `minmax()` [CSS function](#) defines a size range greater than or equal to *min* and less than or equal to *max*

18

Die Visualisierung zeigt den Kerngedanken: Ein Grid wird mit `grid-template-columns` und `grid-template-rows` als Raster aufgespannt, und Elemente werden entweder automatisch platziert oder mit `grid-area` explizit in benannte Zellen eingeordnet. `grid-template-areas` ist der lesbarste Weg, weil die Struktur als ASCII-Art im CSS erkennbar ist — ein Blick auf die Areas-Definition zeigt sofort, wie das Layout aussieht. Das ersetzt Dutzende Zeilen `grid-column/grid-row`-Angaben durch ein Diagramm, das auch nicht-CSS-Experten lesen können. Grid eignet sich besonders für Seitenlayouts, Dashboards, Formulare und alles, was ein klares 2D-Muster hat.

Fluid Grid

Ein fluides Grid ist ein Raster, das **mit der verfügbaren Breite mitwächst**. Die Struktur bleibt gleich, aber die Spalten passen sich an.

Warum das wichtig ist

- Desktop, Tablet und Mobile brauchen nicht drei getrennte Layouts
- Das Raster reagiert auf die verfügbare Breite
- `minmax()` und `auto-fit` helfen, Spalten flexibel zu halten

Beispiel

```
.product-grid {  
  display: grid;  
  gap: 1rem;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
}
```

Was hier passiert

- `auto-fit` füllt die Zeile mit so vielen Spalten wie möglich
- `minmax(220px, 1fr)` sagt: nie kleiner als 220px, sonst flexibel wachsen
- Das ist der Kern eines fluiden, responsiven Grids



Konzeptionell: Grid legt die Struktur fest, fluides Grid macht diese Struktur responsiv.

auto-fit vs auto-fill ist eine subtile Unterscheidung, die oft übersehen wird: auto-fit kollabiert leere Tracks, auto-fill behält sie. Für eine Produktliste mit wenigen Items wollen Sie meistens auto-fit, damit die verbleibenden Karten sich die volle Breite teilen, statt rechts leere Spalten zu hinterlassen. `minmax(220px, 1fr)` ist das Schlüsselmuster: die Spalte wird nie schmaler als 220px (sonst bricht das Design), aber sie wächst bei Bedarf auf gleichen Anteil (1fr). Zusammen ergibt das ein responsives Raster, das auf jedem Viewport ohne Media Queries funktioniert — ein mächtiges Muster, das in der Praxis unterbenutzt ist.

Interpretationsaufgabe: Layout-Entscheidung

Situation: Der Online-Shop bekommt eine neue Produktdetailseite. Links das Produktbild, rechts Beschreibung und Kaufbutton. Auf dem Smartphone soll das Bild oben und der Text darunter erscheinen.

Welches Layoutmodell wählen Sie — und warum?

Speaker notes

Lassen Sie die Studierenden zwei Minuten nachdenken und ihre Entscheidung begründen, bevor Sie zur nächsten Folie gehen. Erwarten Sie die drei üblichen Vorschläge: Flexbox (weil es einfach klingt), Grid (weil es flexibel ist) und zwei separate Layouts (weil es „klar“ ist). Die Aufgabe ist bewusst unterdefiniert — die Begründung ist wichtiger als die Antwort. Wer Flexbox wählt, muss erklären können, wie die Umordnung Bild-oben/Text-unten auf Mobile funktioniert. Wer Grid wählt, muss grid-template-areas und Media Queries erklären können. Wer zwei Layouts wählt, muss begründen, warum der doppelte Aufwand sich lohnt.



Analyse

Option	Bewertung
Flexbox mit flex-wrap	Funktioniert: Items brechen bei schmalen Screens um. Aber: Reihenfolge ist schwer zu steuern.
Grid mit grid-template-areas	Besser: Media Query ändert Areas von "img text" zu "img" "text". Explizite Kontrolle.
Zwei separate Layouts	Zu aufwändig — CSS sollte ein Layout responsiv machen, nicht zwei verwalten.

Grid gewinnt, weil die zweidimensionale Kontrolle und benannte Areas das Layout explizit und wartbar machen. Flexbox wäre für eine einfache Reihe (z.B. Navigationslinks) die bessere Wahl.

Die eigentliche Lektion ist nicht, dass Grid immer gewinnt, sondern dass die Wahl bewusst begründet werden muss. In der Praxis sehen Sie häufig Mischformen: Grid für das Seitenlayout, Flexbox für Navigationsleisten und Karteninhalt. Diese Kombination ist kein Kompromiss, sondern die richtige Anwendung beider Werkzeuge auf ihre jeweiligen Probleme. Wer beide Modelle kennt und verstanden hat, welches Werkzeug wann das richtige ist, schreibt CSS deutlich schneller und sauberer als jemand, der alles mit einem Werkzeug lösen will.

Takeaway

CSS existiert als eigene Sprache, weil dasselbe HTML in **verschiedenen Kontexten** verschieden dargestellt werden muss — Screen, Print, Mobile, Dark Mode. CSS ist **deklarativ**: Man beschreibt Constraints, der Browser berechnet Pixel. Spezifität bestimmt, welche Regel gewinnt. Flexbox und Grid lösen unterschiedliche Verteilungsprobleme (Inhalt verteilen vs. Raster vorgeben). Responsive Design mit Mobile-First ist Voraussetzung. Die wichtigste Grenzfrage: **Kann CSS das allein** (Hover, Transition, Media Query), oder braucht es JavaScript? Nur wenn neuer Zustand erzeugt werden muss, ist JS die richtige Schicht.

Wenn Sie aus diesem Modul *eine Sache* mitnehmen, dann die **Deklarativität** von CSS. Das ist es, was CSS von imperativem UI-Code unterscheidet: Sie beschreiben nicht Schritte, sondern Constraints. Der Browser löst sie auf. Dieses Prinzip skaliert auf alle Bildschirmgrößen, alle Geräte, alle Eingabemodi — ohne dass Sie pro Fall Code schreiben müssen. Wer CSS imperativ denkt („JavaScript setzt Pixel“), verfehlt den Kern der Sprache und kämpft gegen sie. Wer CSS deklarativ denkt („ich beschreibe die Regeln, der Browser macht den Rest“), arbeitet mit der Sprache.

Web APIs — die Schnittstellen des Browsers

Leitfrage: Was sind Web APIs — und warum gehören sie nicht zur JavaScript-Sprache selbst, sondern zum Browser?

Die JavaScript-Sprache (ECMAScript) ist erstaunlich klein: Variablen, Funktionen, Schleifen, Objekte — keine Datei-, Netzwerk- oder DOM-Operationen. Alles, was JavaScript im Browser *praktisch* tun kann, kommt von **Web APIs** — Schnittstellen, die der Browser bereitstellt. Das ist wichtig für die Architektur, weil derselbe JavaScript-Code in unterschiedlichen Umgebungen unterschiedliche APIs hat: im Browser `window` und `fetch`, in Node.js `process` und `fs`, in einem Service Worker weder noch das eine noch das andere.

Web APIs in einem Satz

Web APIs sind Browser-Schnittstellen, die JavaScript zusätzliche Fähigkeiten geben: den DOM verändern, Netzwerke ansprechen, Daten speichern, Gerätefunktionen nutzen oder Medien steuern.

Typische Browser-APIs

- DOM API
- Fetch API
- Web Storage API
- Geolocation API
- Canvas / WebGL
- Web Workers

Warum das wichtig ist

- JavaScript allein beschreibt die Sprache
- Web APIs machen sie im Browser praktisch nutzbar
- viele Web-Funktionen sind erst durch diese APIs möglich

Beispiel: `fetch()` lädt Daten, `localStorage` speichert Zustand, DOM APIs aktualisieren die Oberfläche.

Der Begriff „API“ ist im Web-Kontext doppeldeutig: Er kann eine **Web API** im Sinne einer Browser-Schnittstelle meinen (was dieses Modul behandelt) oder eine **HTTP-API** im Sinne eines serverseitigen Endpunkts (was Vorlesung 04 behandelt). Beide sind APIs im breiten Sinn „Application Programming Interface“. Wenn Sie in diesem Modul von APIs sprechen, meinen Sie Schnittstellen *des Browsers* an den JavaScript-Code. Diese kommen nicht aus ECMAScript selbst — JavaScript als Sprache kennt weder `window`, noch `document`, noch `fetch`. Es sind Erweiterungen, die der Browser zur Laufzeit bereitstellt.

Wofür nutzt man sie?

Aufgabe	Beispiel-API
HTML oder CSS zur Laufzeit ändern	DOM API
Daten vom Server laden	Fetch API
Zustand im Browser speichern	Web Storage API
Standorte abfragen	Geolocation API
Lange Berechnungen auslagern	Web Workers

Kernaussage: Web APIs sind die Schnittstelle zwischen JavaScript und den Fähigkeiten des Browsers.

Grenze: Nicht jede API ist Teil von JavaScript selbst. Viele Funktionen kommen vom Browser, nicht von der Sprache.

Die Unterscheidung ist praktisch relevant: Derselbe JavaScript-Code läuft je nach Umgebung unterschiedlich. Im Browser funktioniert `localStorage`, in Node.js nicht. In Node.js funktioniert `fs.readFile`, im Browser nicht. Wer Isomorphic JavaScript (Code, der auf Server und Client läuft) schreibt, muss diese Unterschiede kennen — und muss manchmal mit Polyfills oder Feature-Detection (`if (typeof window !== 'undefined')`) arbeiten. Moderne Full-Stack-Frameworks wie Next.js abstrahieren diese Trennung teilweise, aber unter der Haube bleibt sie bestehen.

Fetch API im Detail: Request, Response, AbortController

```
const controller = new AbortController();

// Request mit Timeout nach 5 Sekunden abbrechen
const timeout = setTimeout(() => controller.abort(), 5000);

try {
  const response = await fetch('/api/products', {
    method: 'GET',
    headers: { 'Accept': 'application/json' },
    signal: controller.signal,
  });

  if (!response.ok) throw new Error(`HTTP ${response.status}`);

  const products = await response.json();
} catch (err) {
  if (err.name === 'AbortError') console.log('Request abgebrochen');
  else console.error('Netzwerkfehler', err);
} finally {
  clearTimeout(timeout);
}
```

Konzept	Zweck
Request / Response	Objektmodell für HTTP-Nachrichten
headers	Metadaten: Content-Type, Accept, Authorization
AbortController	Laufende Requests abbrechen (Timeout, Navigation)
response.ok	true bei Status 200–299

Der `AbortController` ist eine unterschätzte API. Ohne ihn läuft ein `Fetch-Request` weiter, auch wenn der Nutzer inzwischen weggenavigiert ist oder die Komponente unmounted wurde — der `Response` trifft dann auf DOM-Elemente, die nicht mehr existieren, und löst kryptische Fehler aus. Moderne Frameworks nutzen `AbortController` intern, um Cleanup zu machen. Für eigene `Fetch-Logik` ist er Pflicht, sobald es um Such-Eingaben, Navigation oder Timeouts geht. Das Pattern „Request starten, Abort-Signal merken, beim Abbruch `signal.abort()` aufrufen" ist ein Muster, das Sie in jedem produktiven Frontend-Code sehen werden.

Web Storage: localStorage, sessionStorage, Cookies

API	Lebensdauer	Größe	Zugriff
localStorage	Persistent (bis manuell gelöscht)	~5–10 MB	Nur Client (JS)
sessionStorage	Nur aktuelle Browser-Session	~5–10 MB	Nur Client (JS)
Cookies	Konfigurierbar (Session oder Ablaufdatum)	~4 KB	Client + automatisch an Server
IndexedDB	Persistent	Gigabyte-Bereich	Client (asynchron)

```
// localStorage: einfache Key-Value-Speicherung
localStorage.setItem('cart', JSON.stringify(cartItems));
const cart = JSON.parse(localStorage.getItem('cart') || '[]');

// sessionStorage: nur für die aktuelle Tab-Session
sessionStorage.setItem('wizard-step', '3');

// Cookies: werden bei jedem HTTP-Request mitgesendet
document.cookie = 'theme=dark; max-age=31536000; SameSite=Strict';
```

Wann was?

- **Cookies** für alles, was der **Server wissen muss** (Session-ID, Authentifizierung)
- **localStorage** für Client-State, der **über Sessions** erhalten bleiben soll (Warenkorb, UI-Einstellungen)
- **sessionStorage** für temporären State, der **nur im aktuellen Tab** gilt (Wizard-Fortschritt)
- **IndexedDB** für große Datenmengen oder Offline-Anwendungen



Ein häufiger Fehler: Authentifizierungs-Tokens in `localStorage` zu speichern. Das ist verlockend, weil es einfach ist, aber gefährlich: Jeder JavaScript-Code auf der Seite (inklusive eingebettete Drittanbieter-Skripte und XSS-Angriffe) kann `localStorage` lesen. Cookies mit `HttpOnly` und `SameSite=Strict` sind sicherer, weil sie JavaScript verborgen bleiben und nur vom Server gelesen werden können. Das ist eine der Architektur-Entscheidungen, die Vorlesung 7 (Sicherheit) vertieft. Für Warenkorb und UI-State ist `localStorage` dagegen völlig in Ordnung — kein Angreifer gewinnt etwas durch Kenntnis des Warenkorbinhalts.

History API: Navigation ohne Seiten-Reload

Single-Page Applications navigieren ohne vollständigen Seiten-Reload. Der Browser-Zurück-Button muss trotzdem funktionieren. Die **History API** macht das möglich:

```
// URL ändern, ohne neu zu laden
history.pushState({ page: 'products' }, '', '/products');

// Neue Inhalte fetch'en und rendern
const data = await fetch('/api/products').then(r => r.json());
renderProducts(data);

// Zurück-Button abfangen
window.addEventListener('popstate', (event) => {
  if (event.state?.page === 'products') renderProducts(/* ... */);
});
```

Methode / Event	Zweck
history.pushState(state, '', url)	URL ändern, Eintrag zum Verlauf hinzufügen
history.replaceState(...)	URL ändern, ohne neuen Eintrag
popstate Event	Browser-Zurück-/Vorwärts-Button

Die History API ist der technische Kern jedes Client-Side-Routers — React Router, Vue Router und SvelteKit nutzen sie unter der Haube. Das Problem, das sie löst: Ohne History API würde jede Navigation in einer SPA die URL unverändert lassen, und der Browser-Zurück-Button würde die SPA verlassen. Mit `pushState` bleibt die URL konsistent mit dem UI-Zustand, und der Zurück-Button funktioniert wie erwartet. Wichtige Feinheit: `pushState` löst *kein* `popstate`-Event aus — das Event feuert nur beim Zurück-Navigieren. Eigene Navigationslogik muss also zwei Pfade unterstützen: aktives Navigieren (UI ruft `pushState` + Render auf) und passives Zurück-Navigieren (`popstate`-Event triggert Render).

Web Workers vs Service Workers

Beide laufen in einem **separaten Thread** außerhalb des Haupt-Event-Loops — aber mit unterschiedlichen Zwecken.

Aspekt	Web Worker	Service Worker
Zweck	CPU-intensive Berechnungen auslagern	Netzwerk-Proxy, Offline-Support, Push
Lebensdauer	Gebunden an Tab/Seite	Persistent, auch wenn Tab geschlossen
DOM-Zugriff	Nein	Nein
Fetch-Interception	Nein	Ja — fängt alle Netzwerk-Requests ab
Typischer Einsatz	Bildverarbeitung, Datenanalyse, Kryptographie	Progressive Web Apps, Caching, Push-Benachrichtigungen

```
// Web Worker: teure Berechnung auslagern
const worker = new Worker('compute.js');
worker.postMessage({ data: largeDataset });
worker.onmessage = (e) => displayResult(e.data);

// Service Worker: Fetch abfangen für Offline-Support
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then(cached => cached || fetch(event.request))
  );
});
```



Service Workers sind das Fundament jeder Progressive Web App (PWA). Sie ermöglichen Offline-Betrieb, weil sie Netzwerk-Requests abfangen und aus einem Cache beantworten können — selbst wenn die Verbindung weg ist. Web Workers dagegen lösen ein ganz anderes Problem: Lange Berechnungen (Bildverarbeitung, Parsen großer JSON-Dateien, Kryptographie) blockieren sonst die UI. Im Worker läuft dieser Code in einem echten zweiten OS-Thread, ohne den Hauptthread zu beeinträchtigen. Beide haben *keinen* DOM-Zugriff — sie kommunizieren mit dem Hauptthread über `postMessage`. Das ist wichtig, weil es bedeutet, dass Worker keine UI direkt ändern können; sie liefern nur Daten zurück.

Takeaway

Web APIs sind die **Fähigkeiten**, die der Browser JavaScript zur Verfügung stellt — nicht Teil der Sprache selbst, sondern Erweiterungen. Die wichtigsten Kategorien sind DOM-Manipulation, Netzwerk (Fetch), Speicher (localStorage/Cookies), Hintergrund-Threads (Web/Service Workers) und History. Welche API wofür eingesetzt wird, ist eine Architekturfrage mit Konsequenzen für Sicherheit, Performance und Offline-Fähigkeit.

Wenn Sie aus diesem Modul *eine Sache* mitnehmen, dann dass die Grenze zwischen Sprache und Plattform wichtig ist. JavaScript allein kann erstaunlich wenig — es sind die Web APIs, die moderne Web-Anwendungen möglich machen. Wer diese Grenze versteht, versteht warum derselbe Code auf Node.js anders läuft als im Browser, warum Polyfills existieren, warum Feature-Detection besser ist als Browser-Detection, und warum manche „JavaScript-Features“ eigentlich Browser-Features sind.

Das Zusammenspiel im Browser

Leitfrage: Wie arbeiten HTML, CSS und JavaScript im Browser zusammen — und warum ist das Verständnis dieser Pipeline für Performance und Architektur entscheidend?

Bisher haben wir die drei Technologien getrennt betrachtet. Im Browser laufen sie aber in einer **gemeinsamen Pipeline**: HTML wird zum DOM, CSS zum CSSOM, dann fließen beide in den Render Tree, und JavaScript kann jederzeit dazwischenfunken. Wer Performance-Probleme im Frontend beheben oder von vornherein vermeiden will, muss diese Pipeline kennen — die meisten langsamen Webseiten sind nicht *durch* eine Technologie langsam, sondern *zwischen* den Technologien.

Was passiert, wenn Sie „In den Warenkorb" klicken?

Aufgabe: Skizzieren Sie den Ablauf: Welche Technologie (HTML, CSS, JavaScript) ist in welchem Schritt beteiligt? Was passiert sofort, was asynchron? Was sieht der Nutzer wann?

Speaker notes

Diese Aufgabe sollte das Zusammenspiel aller bisherigen Inhalte auf einem konkreten Szenario zeigen. Lassen Sie die Studierenden einige Minuten nachdenken und sammeln Sie Vorschläge an der Tafel. Erwarten Sie unterschiedliche Detailtiefen: manche werden nur sagen „der Button wird geklickt, dann kommt das Produkt in den Warenkorb“, andere werden schon auf den Event-Handler, den Fetch-Aufruf und das optimistische UI-Update eingehen. Beides ist richtig — die nächste Folie löst gezielt in sechs Schritten auf, die *alle* Technologien sichtbar machen.



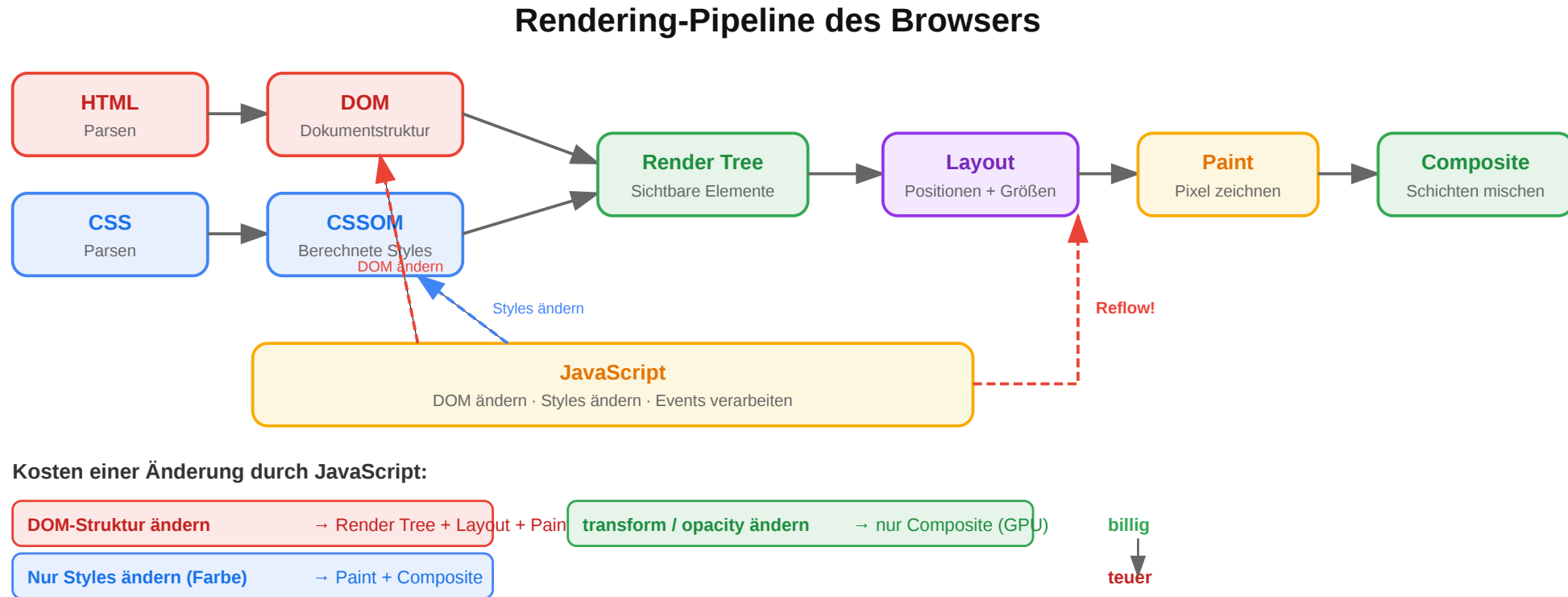
Ablauf (Skizze)

1. Nutzer klickt den Button → JavaScript reagiert auf das Event.
2. JavaScript aktualisiert DOM-Zustände (Badge, Button-Text).
3. CSS/Layout berechnet neues Rendering, der Nutzer sieht sofort Feedback.
4. JavaScript sendet asynchron POST `/api/cart` an den Server.
5. Server antwortet, JavaScript aktualisiert DOM erneut.
6. Browser führt Layout und Paint erneut aus.

Schlüsselbeobachtung: Das UI reagiert **sofort** (optimistisches Update), der Server wird **asynchron** kontaktiert. Das ist nur möglich, weil JavaScript non-blocking arbeitet.

Dieser sechsstufige Ablauf macht das Zusammenspiel der drei Technologien sichtbar: HTML liefert die Button-Struktur, CSS die Darstellung, JavaScript das Verhalten und die Netzwerk-Kommunikation, und der Browser orchestriert Layout und Paint. Beobachten Sie, wie viele Schritte *sofort* ablaufen und wie wenige *asynchron* sind — das Verhältnis ist bewusst so gewählt: Der Nutzer soll nie mehr als 100ms auf Feedback warten (das ist die Grenze, ab der Menschen eine Reaktion als „verzögert“ wahrnehmen). Alles andere läuft im Hintergrund.

Die Rendering-Pipeline des Browsers



Die Pipeline arbeitet auf zwei internen Strukturen: dem **DOM** für HTML und dem **CSSOM** für CSS. Erst daraus erzeugt der Browser den Render Tree und rechnet Layout, Paint und Compositing aus.

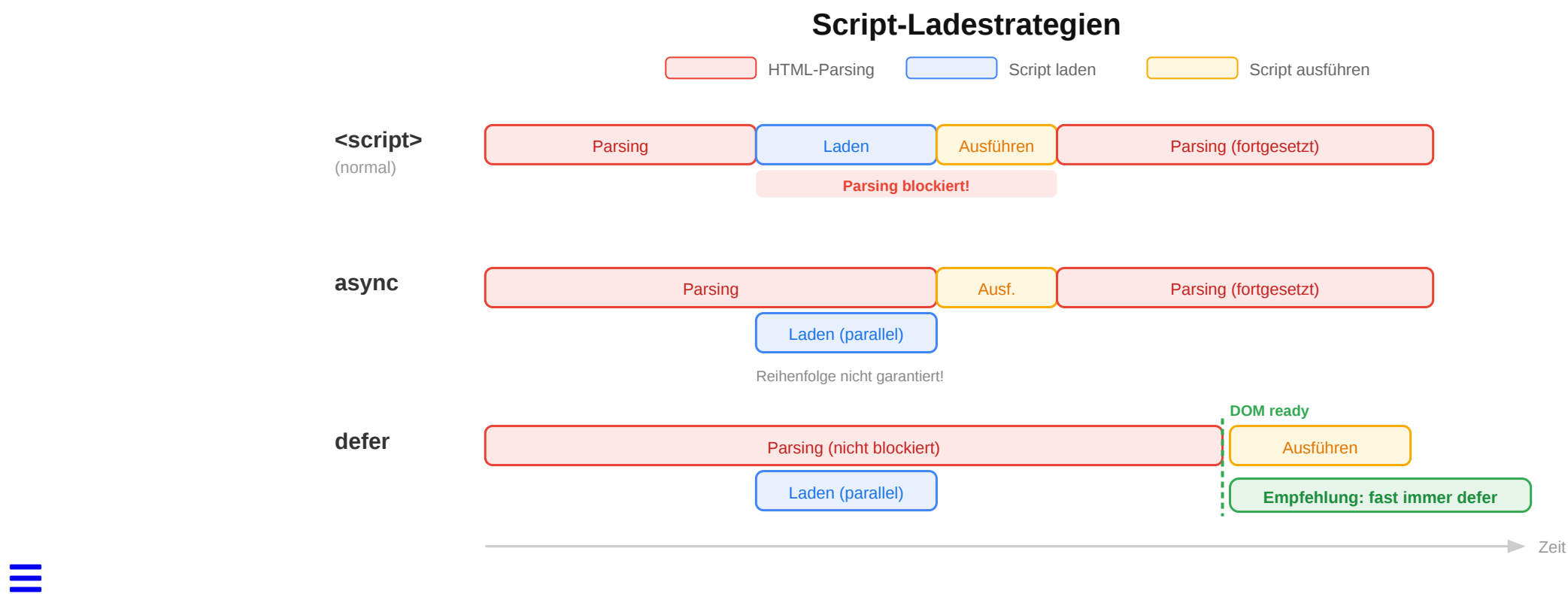
Die Rendering-Pipeline ist deterministisch und läuft in sechs Phasen: (1) HTML parsen → DOM, (2) CSS parsen → CSSOM, (3) Render Tree aufbauen (DOM + CSSOM zusammenführen, dabei unsichtbare Elemente ausfiltern), (4) Layout (Berechnung von Position und Größe jedes Elements), (5) Paint (Pixel zeichnen), (6) Composite (Layer zusammenstellen). Bei jeder Änderung am DOM oder CSS fragt der Browser: Wie weit muss ich zurück in die Pipeline? Nur Farbe geändert? → nur Paint + Composite. Position geändert? → Layout + Paint + Composite (Reflow). Nur Transform oder Opacity? → nur Composite (am billigsten). Diese Unterscheidung ist der Schlüssel zu 60-FPS-Animationen.

Warum ist diese Seite langsam?

Aufgabe: Eine Produktseite des Online-Shops lädt in 4,2 Sekunden. Der Entwickler hat den HTML-Head so strukturiert:

```
<head>
<script src="analytics.js"></script>
<script src="app.js"></script>
<link rel="stylesheet" href="styles.css">
<script src="chat-widget.js"></script>
</head>
```

Was ist das Problem — und wie würden Sie es lösen?



Lassen Sie die Studierenden das Problem selbst finden, bevor Sie auflösen. Der springende Punkt: Jedes `<script>` ohne `defer` oder `async` stoppt den HTML-Parser. Der Browser kann nichts rendern, weil das Script potenziell `document.write` aufrufen oder den DOM ändern könnte. Drei Scripts hintereinander bedeuten drei sequenzielle Blockaden. Und weil das CSS *nach* den Scripts kommt, wartet der Browser auf die Scripts, bevor er das Stylesheet überhaupt lädt. Ergebnis: Der Nutzer sieht eine weiße Seite, bis alle Ressourcen geladen sind.

Analyse

Problem	Ursache	Lösung
Scripts blockieren Parsing	<script> ohne defer/async stoppt den HTML-Parser	<script src="app.js" defer>
CSS kommt nach Scripts	Browser wartet auf Scripts, bevor er CSS sieht	CSS vor Scripts im <head>
Alles synchron geladen	3 Scripts + 1 CSS = sequenzielle Blockade	defer für alle Scripts, CSS zuerst

Optimierte Version

```
<head>
<link rel="stylesheet" href="styles.css">
<script src="app.js" defer></script>
<script src="analytics.js" defer></script>
<script src="chat-widget.js" defer></script>
</head>
```

defer = Script wird **nach** dem Parsen ausgeführt, **in Reihenfolge**. Das ist fast immer die richtige Wahl.

Der Unterschied zwischen `defer` und `async`: `defer` lädt parallel zum HTML-Parsing und führt nach dem Parsing in der Reihenfolge des Quelltexts aus — deshalb sicher für Scripts mit Abhängigkeiten. `async` lädt ebenfalls parallel, führt aber sofort beim Eintreffen aus, in unvorhersehbarer Reihenfolge — geeignet für unabhängige Scripts wie Analytics, die keinen anderen Code brauchen. Als Faustregel: `defer` ist die bessere Default-Wahl. `async` nur für wirklich unabhängige, `feuer-und-vergiss`-Scripts. Eine dritte Alternative ist, Scripts ans Ende des `<body>` zu setzen — funktioniert ebenfalls, ist aber weniger deklarativ und trennt den Head von seiner Konfiguration.

Performance: Die häufigsten Fehler

Problem	Ursache	Lösung
Render-Blocking CSS	Browser wartet auf gesamtes CSS, bevor er etwas anzeigt	Critical CSS inline, Rest asynchron laden
Parser-Blocking JS	<script> stoppt HTML-Parsing	defer oder async Attribut
Layout Thrashing	JavaScript liest und schreibt abwechselnd DOM-Properties	Reads bündeln, dann Writes bündeln
Große JS-Bundles	Zu viel JavaScript auf einmal geladen	Code Splitting, Lazy Loading
Unoptimierte Bilder	5 MB Produktbilder ohne Kompression	WebP/AVIF, responsive srcset, Lazy Loading

Kernprinzip

Performance-Probleme im Web entstehen fast immer an der **Schnittstelle** von HTML, CSS und JavaScript — nicht innerhalb einer einzelnen Technologie. Deshalb muss man die Pipeline verstehen.

Layout Thrashing ist die am häufigsten übersehene Falle: Wenn JavaScript abwechselnd eine DOM-Property liest (`element.offsetHeight`) und eine schreibt (`element.style.height = ...`), muss der Browser *bei jedem Lesen* Layout neu berechnen, weil er nicht weiß, ob die vorherige Schreiboperation ihn invalidiert hat. Das Ergebnis: Aus einer 10-ms-Schleife wird eine 500-ms-Schleife. Lösung: Alle Reads zuerst, dann alle Writes. Die `requestAnimationFrame`-API ist der richtige Ort für DOM-Updates, weil der Browser dort weiß, dass danach ein Frame gerendert wird und er Batching betreiben kann.

Core Web Vitals: Wie Google Performance misst

Google misst Performance anhand dreier Nutzer-zentrierter Metriken, die auch ins Suchranking einfließen:

Metrik	Was es misst	Ziel
LCP — Largest Contentful Paint	Zeit, bis das größte sichtbare Element gerendert ist	< 2,5 s
INP — Interaction to Next Paint	Verzögerung zwischen Klick/Tap und sichtbarer Reaktion	< 200 ms
CLS — Cumulative Layout Shift	Wie stark sich das Layout nach Laden verschiebt	< 0,1

Typische Verursacher

- **Schlechter LCP:** große, unoptimierte Hero-Bilder; render-blocking CSS; langsamer Server
- **Schlechter INP:** lange Haupt-Thread-Blockaden durch JavaScript
- **Schlechter CLS:** Bilder ohne width/height; nachgeladene Werbung oder Fonts verschieben Inhalt

Warum das architektonisch wichtig ist

Core Web Vitals sind keine Laborwerte — sie werden am **echten Nutzer** gemessen (Real User Monitoring über Chrome UX Report). Eine Seite kann im Labor schnell wirken und in der Praxis trotzdem schlechte Werte haben, wenn sie auf langsamen Geräten oder Netzwerken läuft.

Core Web Vitals sind seit 2021 ein Ranking-Faktor bei Google — schlechte Werte können also direkte wirtschaftliche Konsequenzen haben. Die drei Metriken sind bewusst so gewählt, dass sie die drei wichtigsten Nutzerwahrnehmungen abbilden: „Ist die Seite schon da?“ (LCP), „Reagiert sie, wenn ich klicke?“ (INP), „Springt der Inhalt unter meinen Fingern weg?“ (CLS). Alle drei hängen direkt mit der Rendering-Pipeline zusammen: LCP ist ein Pipeline-Durchlauf, INP ist Hauptthread-Verfügbarkeit, CLS ist Reflow nach initialem Layout. Wer die Pipeline verstanden hat, weiß automatisch, wie man diese Metriken verbessert.

Wie hat sich das Zusammenspiel verändert?

Ära	Muster	Zusammenspiel
Klassisch (2000er)	Server rendert HTML, CSS als separate Datei, JS für kleine Effekte	Strikte Trennung: HTML-Datei + CSS-Datei + JS-Datei
jQuery-Ära (2006–2015)	HTML vom Server, JS manipuliert DOM direkt	DOM als zentrale Schnittstelle
Komponentenbasiert (ab 2013)	React/Vue/Angular: HTML + CSS + JS pro Komponente	Trennung nach Verantwortung , nicht nach Technologie
Server-First (ab 2020)	Next.js/SvelteKit: Server rendert, Client hydriert	Erst HTML vom Server, dann JavaScript macht es interaktiv

Was bedeutet „Client hydriert“?

Der Server liefert bereits fertiges HTML an den Browser. Danach lädt JavaScript nach und **bindet Verhalten an dieses vorhandene HTML**:

- Event-Listener werden registriert
- Komponenten bekommen ihren interaktiven Zustand
- die Seite wird nutzbar, ohne dass der Browser alles neu rendern muss

Kurz: Hydration bedeutet, dass der Client das serverseitig gerenderte HTML „zum Leben erweckt“.

Diskussionsfrage

Die klassische Web-Lehre sagt: „Trenne HTML, CSS und JavaScript in separate Dateien." React bündelt alle drei **pro Komponente**. Ist das ein Widerspruch zum Prinzip der Separation of Concerns — oder eine bessere Umsetzung davon?

Die Evolution des Web-Frontends lässt sich in vier Ären einteilen, und jede hatte ein anderes Verständnis von Separation of Concerns. In den 2000ern bedeutete „trennen“ = separate Dateien pro Technologie. In der jQuery-Ära wurde JS zur zentralen Steuerung, HTML und CSS blieben auf dem Server. Mit React (2013) kam der Paradigmenwechsel: Trennung nach fachlicher Verantwortung statt nach Technologie. Und seit 2020 pendelt die Branche wieder zurück — Server-First, aber mit Client-Hydration, um das Beste aus beiden Welten zu verbinden. Die Diskussionsfrage hat keine eindeutige Antwort: Beide Sichten sind legitim, und die richtige Wahl hängt vom Projekt ab.

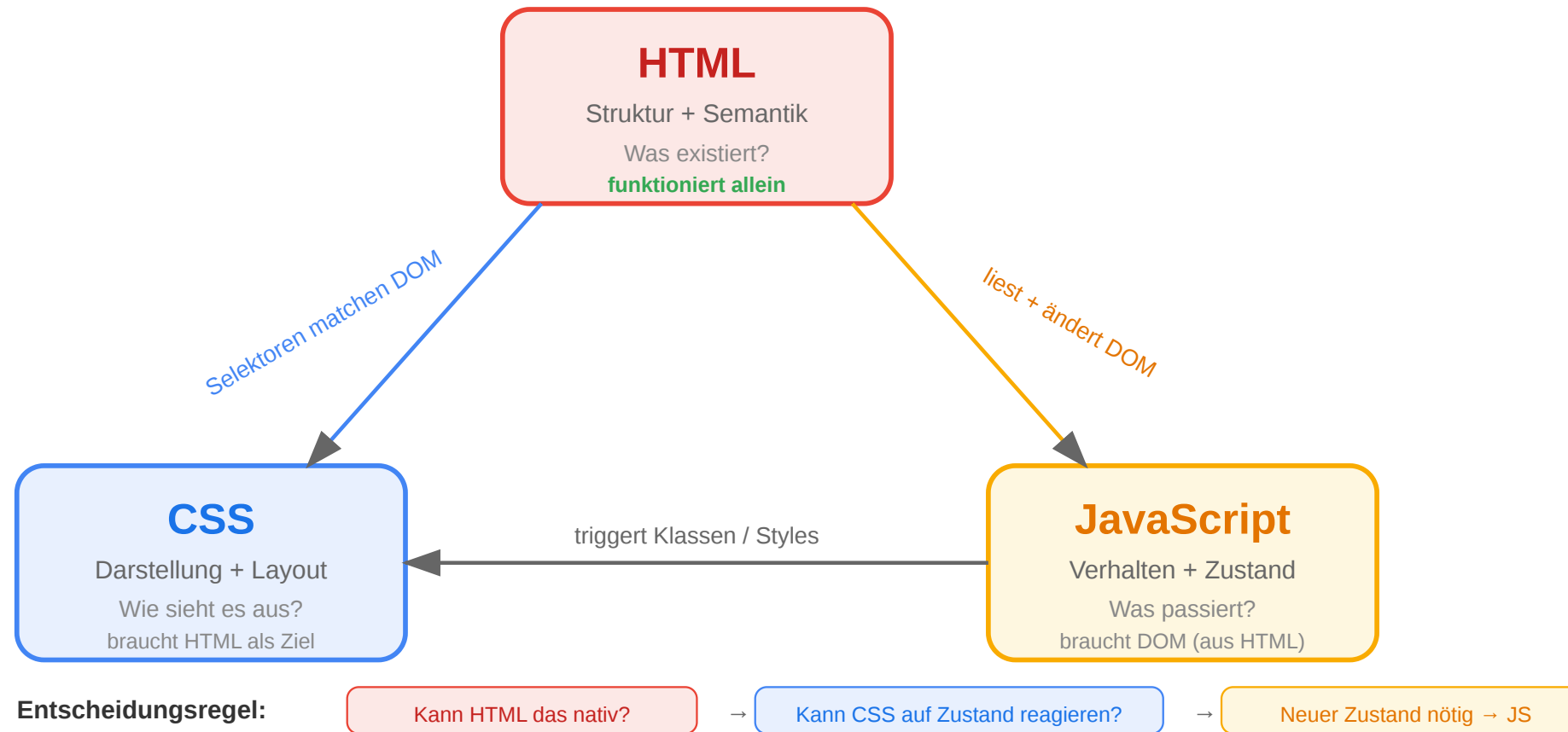
Einordnung

Beides stimmt: Die Trennung bleibt — aber auf einer anderen Ebene. Statt nach Technologie (HTML/CSS/JS) wird nach **fachlicher Verantwortung** getrennt (Produktkarte, Warenkorb, Navigation). Jede Komponente kapselt ihr Markup, ihren Stil und ihr Verhalten.

Diese Einordnung erklärt, warum beide Sichten „richtig“ sind. Separation of Concerns ist ein abstraktes Prinzip — es sagt nur, dass zusammengehöriger Code zusammenbleiben und unzusammenhängender getrennt werden soll. Was „zusammengehörig“ bedeutet, ist aber Auslegungssache. In den 2000ern war das die Technologie, heute ist es die Fachlichkeit. Beide Interpretationen entsprechen dem Prinzip — sie sind nur unterschiedliche Antworten auf die Frage „was hängt zusammen“. Wer das einmal verstanden hat, wird in zukünftigen Technologie-Debatten nicht mehr auf falsche Dichotomien hereinfallen.

Das Technologie-Dreieck: Zusammenfassung

Das Technologie-Dreieck des Webs



HTML, CSS und JavaScript sind ein eng verzahntes System. Der Browser verarbeitet sie in einer festen Pipeline, die JavaScript jederzeit beeinflussen kann — mit Konsequenzen für Performance und Architektur. Moderne Frameworks organisieren das Zusammenspiel in Komponenten statt in Dateien. Das grundlegende Modell von DOM, Events und Rendering bleibt dasselbe.

Das Technologie-Dreieck ist die zentrale Metapher dieser Vorlesung: drei Schichten mit klaren Verantwortungen, die im Browser zusammenfließen. Wer diese drei kennt und versteht, wie sie in der Rendering-Pipeline interagieren, hat die Grundlage für jede Frontend-Architektur — egal ob Vanilla JavaScript, React, Svelte oder ein Framework, das es in zehn Jahren noch nicht gibt. Das konkrete Werkzeug wechselt, das Modell bleibt.

Wrap-Up

- **HTML** definiert Struktur und Semantik — die Grundlage für Barrierefreiheit, SEO und maschinelle Verarbeitung.
- **JavaScript** macht den Browser programmierbar — über DOM, Events und asynchrone Kommunikation mit dem Server.
- **CSS** trennt Darstellung von Struktur — Flexbox und Grid lösen das Layout-Problem, Responsive Design das Geräteproblem.
- Die drei Technologien sind **eng verzahnt**: Der Browser verarbeitet sie in einer Pipeline (Parse → Render Tree → Layout → Paint), die JavaScript jederzeit beeinflussen kann.

Die nächste Vorlesung vertieft das **Protokoll** (HTTP) und das **API-Design** (REST), das die Brücke zwischen Frontend und Backend bildet.

Zusammenfassend: Diese Vorlesung hat bewusst keine umfassende Abhandlung zu HTML, CSS oder JavaScript geliefert — das würde jede einzelne Technologie ein ganzes Semester füllen. Statt Syntax haben wir **Architektur** behandelt: warum HTML semantisch sein muss, warum CSS deklarativ ist, warum JavaScript single-threaded läuft, wie die Technologien im Browser zusammenspielen. Wer diese Konzepte mitnimmt, kann jede konkrete Frontend-Technologie schnell lernen. Wer nur Syntax auswendig lernt, muss bei jedem neuen Framework bei null anfangen. In der nächsten Vorlesung geht es auf die andere Seite der Leitung: HTTP als Protokoll zwischen Browser und Server, und REST als API-Stil darauf.

Legacy-Quellen:

- `slides/04_02_WebEng_WebTechnologies-HTML-5.pptx`
- `slides/04_03_WebEng_WebTechnologies-CSS3.pptx`
- JavaScript-Inhalte sind neu erstellt (kein Legacy-Material vorhanden)

